HAN'S ROBOT
大族机器人

Elfin 系列协作型机器人

ROS&Elfin基本使用教程

（Elfin 系列机器人通用）



目录

第一章 概括 1

1.1. 编写目的 1

1.2. 注意事项 1

1.3. 名词解析 1

第二章 ROS 基本教程 2

2.1. ROS 简介 2

2.1.1. ROS 是什么 2

2.1.2. ROS 历史发展与版本 2

2.1.3. ROS 设计目标与特点 4

2.1.4. ROS 优缺点 5

2.1.5. ROS 参考资料 5

2.2. 安装教程 6

2.2.1. 添加软件源 6

2.2.2. 设置秘钥 6

2.2.3. 安装 ROS 6

2.2.4. 初始化 6

2.2.5. 设置环境变量 7

2.2.6. 安装基础功能包 7

2.2.7. 安装结果测试 7

2.3. ROS 架构 8

2.3.1. ROS 文件系统组成 8

2.3.2. ROS 文件系统相关命令 9

2.3.3. ROS 计算图 11

2.4. ROS 节点和消息 12

2.4.1. ROS 节点 12

2.4.2. ROS 消息 13

2.5. ROS 通信机制 15

2.5.1. 话题通信(Topic) 15

2.5.2. 服务通信(Service) 17

2.5.3. 动作通信(Action) 19

2.5.4. 常用命令 20

第三章 ELFIN-ROS 基本教程 24

3.1. 环境配置 24

3.1.1. 安装环境 24

3.1.2. 仿真环境测试 25

3.2. 通过 ROS 控制真实 ELFIN 机器人 27

3.2.1. 分支说明 27

3.2.2. 整体连接过程 27

3.2.3. 硬件连接 28

3.2.4. 软件配置 29

3.2.5. 连接机器人 31

3.2.6. 基本控制使用 35

3.3. 基本接口介绍 36

3.3.1. 话题接口 37

3.3.2. 服务接口 38

3.4. 常用接口使用&DEMO 39

3.4.1. 控制接口 39

3.4.2. 参数/状态接口 45

3.4.3. IO 接口 47

第四章 常见问题的排查与解决方法 49

4.1. 连接问题 49

4.1.1. 没有扫描到从站 49

4.1.2. 连接成功，但一直会有打印信息 49

第一章 概括

1.1.编写目的

1. 为使用 ROS&Elfin 的人员提供基本使用说明和 ROS 基本入门教程。
2. 提供 ROS 下 Elfin 机器人的常用接口示例，仅限于 C++与 Python

1.2.注意事项

1. 本文档基于 Ubuntu18.04 ROS-Melodic 环境，对于其他版本请自行查阅版本差异，并修改相关示例
2. 对于 ROS 初学者，本文仅提供 ROS 的基础说明，对于更多的使用和相关资料请自行查阅
3. 本文档基于 Ubuntu18.04 ROS-Melodic 环境，对于其他版本在使用本文提到的相关安装命令时请自行修改版本名，防止出现安装错误或无法定位到软件包等问题
4. 本文档会对 ROS 初学者与不熟悉 Elfin 相关 ROS 接口的用户有所帮助。

1.3.名词解析

专用名称	解释
ACS	关节坐标，单位为度(°)
PCS	空间坐标，单位为毫米(mm)、度(°)
TCP	系统默认的工具坐标系
UCS	用户坐标系
Base	系统默认的用户坐标系
Tool	系统默认的工具坐标系

第二章 ROS 基本教程

2.1.ROS 简介

ROS 简介，主要介绍 ROS 相关基本内容

2.1.1.ROS 是什么

机器人操作系统(Robot Operating System, ROS)是一个应用于机器人上的操作系统，它操作方便、功能强大，特别适用于机器人这种多节点多任务的复杂场景。

ROS 是一个适用于机器人编程的框架， 这个框架把原本松散的零部件耦合在了一起， 为他们提供了通信架构。 ROS 虽然叫做操作系统， 但并非 Windows、 Mac 那样通常意义的操作系统， 它只是连接了操作系统和你开发的 ROS 应用程序， 所以它也算是一个中间件， 基于 ROS 的应用程序之间建立起了沟通的桥梁， 所以也是运行在 Linux 上的运行环境， 在这个环境上基于 ROS 的系统框架对机器人的感知、 决策、 控制算法可以更好的组织和运行。

2.1.2.ROS 历史发展与版本

2007 年诞生于斯坦福大学，2010 年正式开源，发布 1.0 版本

ROS 发行版本按照英文字母顺序命令，目前已经发布了 ROS1 的最终版本：Noetic，在生命周期终止后不再进行维护，Noetic 版本之前默认使用 Python2，noetic 支持 Python3。

ROS1 的所有版本如下表所示，绿色代表该版本还在维护中，灰色代表该版本已经停止维护。目前对于 ROS 版本的选择比较推荐使用 Indigo、Kinetic、Melodic 和 Noetic 四个版本。

发行名称	发行日期	生命周期终止日期
ROS Noetic Ninjemys(Recommended)	May 23rd, 2020	May, 2025(Focal EOL)
ROS Melodic Morenia	May 23rd, 2018	May, 2023(Bionic EOL)
ROS Lunar Loggerhead	May 23rd, 2017	May, 2019
ROS Kinetic Kame	May 23rd, 2016	April, 2021(Xenial EOL)
ROS Jade Turtle	May 23rd, 2015	May, 2017
ROS Indigo Igloo	July 22nd, 2014	April, 2019(Trusty EOL)
ROS Hydro Medusa	September 4th, 2013	May, 2015
ROS Groovy Galapagos	31-Dec-12	July, 2014
ROS Fuerte Turtle	23-Apr-12	--
ROS Electric Emys	30-Aug-11	--
ROS Diamondback	2-Mar-11	--
ROS C Turtle	2-Aug-10	--
ROS Box Turtle	2-Mar-10	--

2.1.3.ROS 设计目标与特点

2.1.3.1. 设计目标

① 提高机器人研发中的软件复用率

② 框架中的每个功能模块能够被单独设计、编译、运行时松耦合方式结合

③ 功能模块都封装于独立的功能包或元功能包，便于社区共享与分发

④ ROS 由核心通信机制、开发工具、应用功能和生态系统四个部分构成

⑤ 为机器人开发提供了硬件抽象、底层驱动、消息传递、程序管理、应用原型等机制，整合了第三方工具及库文件，便于用户快速完成机器人应用的建立、编写和多机整合等

2.1.3.2. 特点

① 点对点的设计

ROS 中的每一个进程都以一个节点的形式运行，可分布于不同的主机，用于分散计算压力和协同工作。

② 多语言支持

ROS 中自定义了一套消息格式和通信接口来实现各个模块间的消息传递，各个模块可以使用不同的编程语言实现，目前使用最广泛的为 C++和 Python，同时支持 C、C#、JAVA 等多种语言。

③ 架构精简、集成度高

每个功能节点可以单独编译，集成了众多开源项目，使用统一的接口，提高软件复用性。

④ 组件化工具包丰富

ROS 中提供了多种工具包以方便用户进行可视化、仿真、数据记录和调试。

3D 可视化工具：Rviz

物理仿真环境：Gazebo

数据记录工具：Rosbag

Qt 工具箱：rqt_*

⑤ 免费并且开源

ROS 拥有 ROS-wiki 社区，可以查阅功能包参数和提问等。

ROS 开发的软件包已经达到数千万个，相关的机器人已经多达上千款。

ROS 遵从 BSD 协议，对个人和商业应用及修改完全免费。

2.1.4.ROS 优缺点

优点	缺点
提供工具、框架和功能	通信的实时性有限
方便移植	系统稳定性尚不满足工业级需求
庞大的用户群体	安全性上没有防护措施
免费开源	仅支持 Linux(Ubuntu)

2.1.5.ROS 参考资料

官网：ROS: Home

wiki：Documentation - ROS Wiki

问答：Questions - ROS Answers: Open Source Q&A Forum

中文入门教程：cn/ROS/Tutorials - ROS Wiki

2.2.安装教程

ROS 安装教程，以 Ubuntu18.04+Melodic 为基础

2.2.1.添加软件源

sudo sh -c 'echo "deb http://packages.ros.org/ros/ubuntu $(lsb_release -sc) main" > /etc/apt/sources.list.d/ros-latest.list'

2.2.2.设置秘钥

sudo apt-key adv --keyserver 'hkp://keyserver.ubuntu.com:80' --recv-key C1CF6E31E6BADE8868B172B4F42ED6FBAB17C654

2.2.3.安装 ROS

sudo apt update && sudo apt install ros-melodic-desktop-full

2.2.4.初始化

按照官方说明进行初始化可能会出现初始化失败的情况，主要原因是境外资源被屏蔽，国内需要翻墙。此时可以通过使用国内初始化工具进行初始化。
官方初始化：

① sudo apt install python-rosdep
② sudo rosdep init
③ rosdep update

国内初始化：
① sudo pip install rosdepc
② sudo rosdepc init
③ rosdepc update

2.2.5.设置环境变量

① echo "source /opt/ros/melodic/setup.bash" >> ~/.bashrc
② source ~/.bashrc

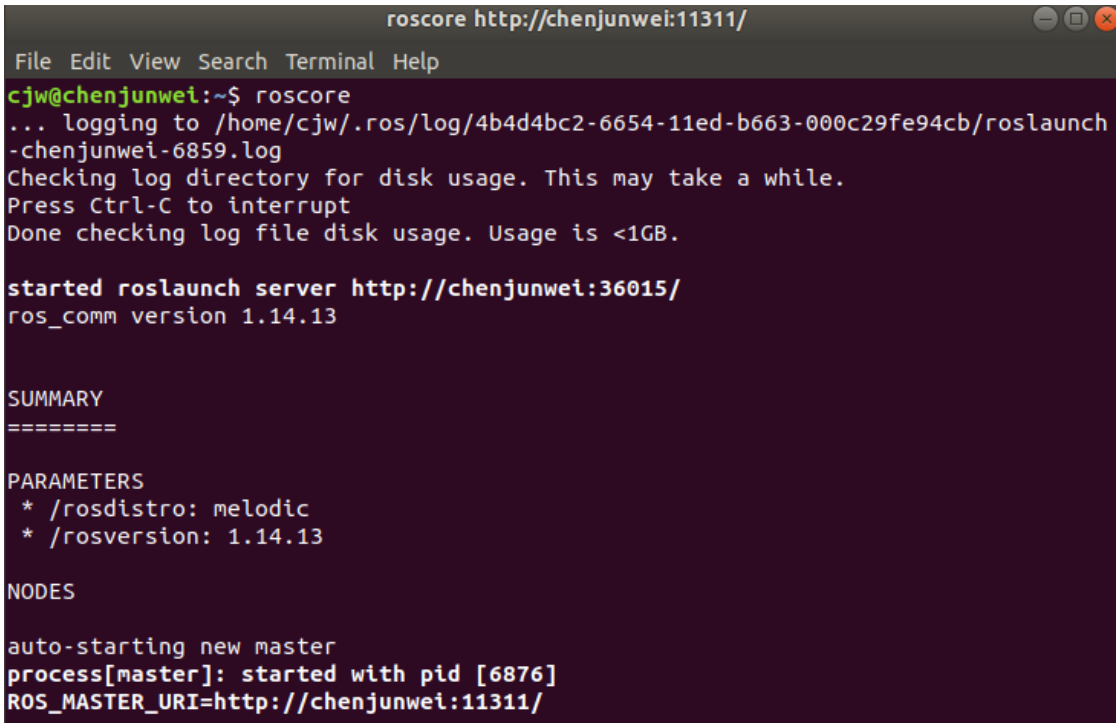
2.2.6.安装基础功能包

sudo apt install python-rosdep python-rosinstall python-rosinstallgenerator pythonwstool build-essentia

2.2.7.安装结果测试

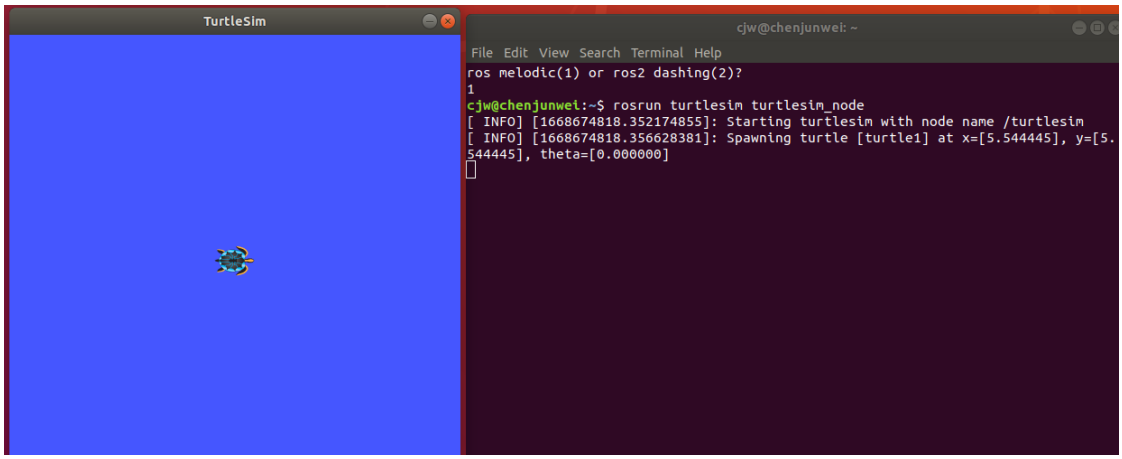
① 启动 roscore

打开终端输入：roscore



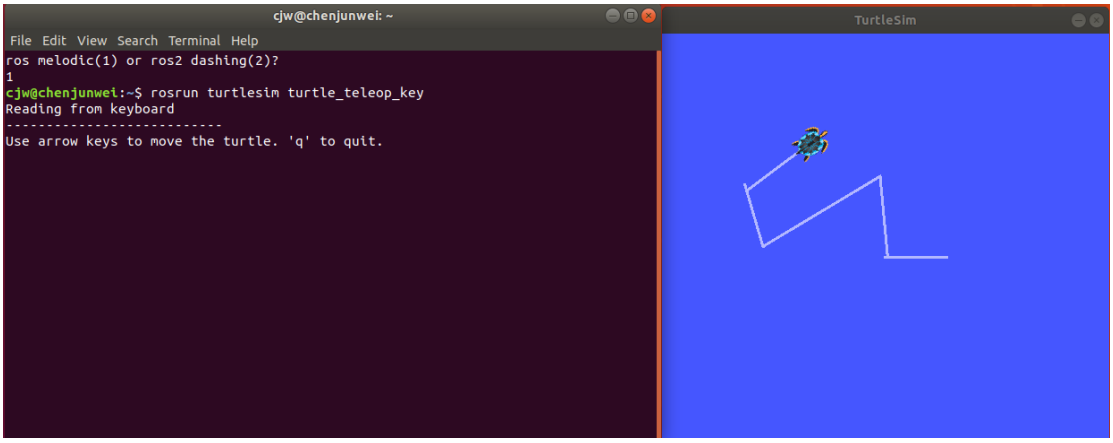
② 启动小海龟

打开终端输入：rosrun turtlesim turtlesim_node



③ 启动键盘控制

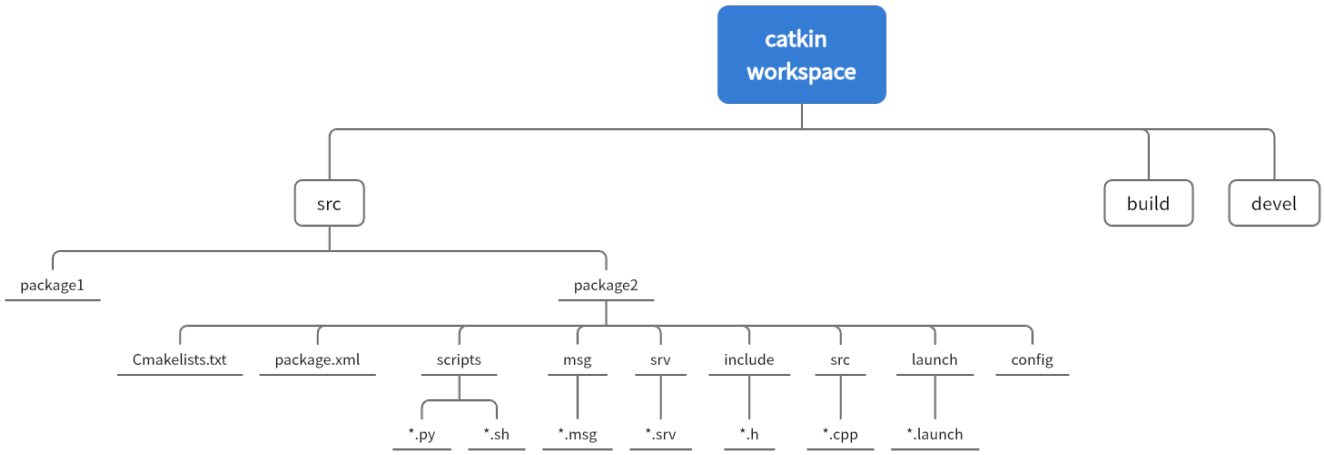
打开终端输入：rosrun turtlesim turtle_teleop_key,通过方向键可以控制小海龟进行移动



2.3.ROS 架构

2.3.1.ROS 文件系统组成

ROS 文件系统是指 ROS 在硬盘上的源代码组织形式，大致结构如下图所示



具体解释：

WorkSpace --- 自定义的工作空间

|--- build:编译空间，用于存放CMake和catkin的缓存信息、配置信息和其他中间文件。

|--- devel:开发空间，用于存放编译后生成的目标文件，包括头文件、动态&静态链接库、可执行文件等。

|--- src: 源码

|-- package：功能包(ROS基本单元)包含多个节点、库与配置文件，包名所有字母小写，只能由字母、数字与下划线组成

|-- CMakeLists.txt 配置编译规则，比如源文件、依赖项、目标文件

|-- package.xml 包信息，比如:包名、版本、作者、依赖项...(以前版本是manifest.xml)

|-- scripts 存储python文件

|-- src 存储C++源文件

|-- include 头文件

|-- msg 消息通信格式文件

|-- srv 服务通信格式文件

|-- action 动作格式文件

|-- launch 可一次性运行多个节点

|-- config 配置信息

|-- CMakeLists.txt: 编译的基本配置

2.3.2.ROS 文件系统相关命令

ROS 的文件系统本质上还是操作系统文件，可以使用 Linux 命令操作文件，但 ROS 上专门提供了一些类似于 Linux 的命令去操作 ROS 的 相关文件以提高用户的 ROS 体验，相比于 Linux 原生命令会比较简介、高效。此处对 ROS 中文件的增删查改和执行进行简单描述。

2.3.2.1. 增

创建工作空间：

```
mkdir -p /catkin_ws/src
```

进入 src 文件夹：

cd ~/catkin_ws/src

初始化文件夹：

catkin_init_workspace

编译工作空间：

cd ~/catkin_ws/

catkin_make

自定义包名、依赖包，创建新的 ROS 功能包：

catkin_create_pkg

安装 ROS 功能包：

sudo apt install ***

2.3.2.2. 删

删除某个功能包：

sudo apt purge ***

2.3.2.3. 查

列出所有功能包：

rospack list

查找某个功能包是否存在，如果存在就返回安装路径：

rospack find 包名

进入某个功能包：

roscd 包名

列出某个包下的文件：

rosls 包名

搜索某个功能包：

apt search ***

2.3.2.4. 改

修改功能包文件(需要安装 vim):

rosed 包名 文件名

2.3.2.5. 执行

roscore：是 ROS 的系统先决条件节点和程序的集合，必须先运行 roscore 才能使 ROS 节点进行通信。

rosrun 包名 可执行文件名：运行 ROS 指定的节点。

roslaunch 包名 launch 文件名：执行某个功能包下的 launch 文件。

2.3.3.ROS 计算图

ROS 的程序运行后会产生很多节点，每个节点的关系错综复杂，为了方便用户理清这一关系，ROS 中提供了 rqt_graph 工具显示当前系统中的 ROS 节点运行情况和每个节点之间的关系。

启动命令：rosrun rqt_graph rqt_graph

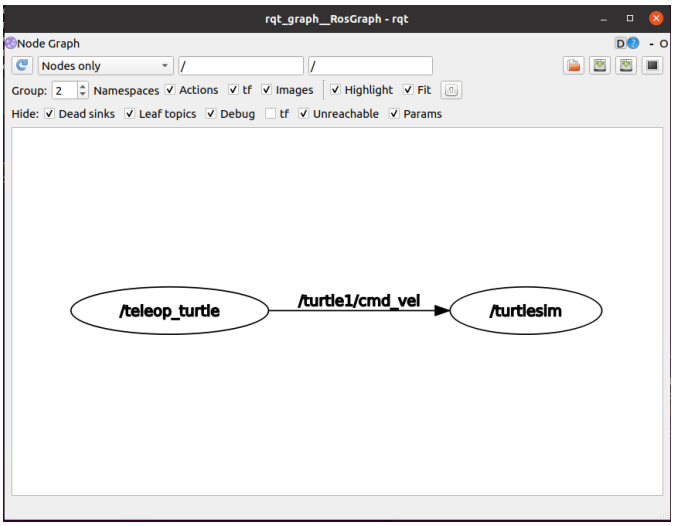
示例:

启动 roscore：roscore

启动 ROS 内置的小乌龟用例：rosrun turtlesim turtlesim_node

启动键盘控制用例：rosrun turtlesim turtle_teleop_key

启动 rqt_graph：rosrun rqt_graph rqt_graph



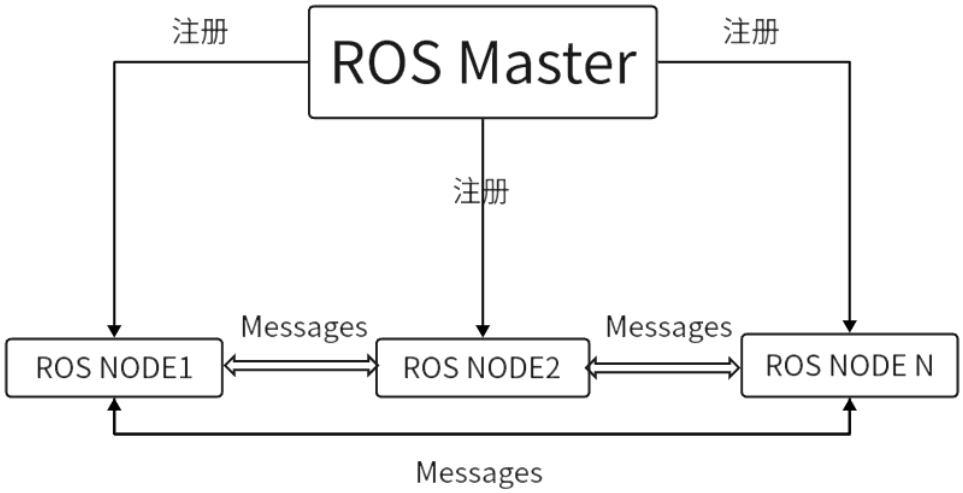
2.4.ROS 节点和消息

ROS 节点和消息作为 ROS 中最基础的两部分内容，可以说 ROS 是基于节点开发的，而节点与节点之间是通过传递消息形式进行通讯，这两部分支撑了整个 ROS 系统。

2.4.1.ROS 节点

ROS 节点是进行运算任务的进程，也可以看做是一个软件模块，ROS 系统是由多个节点组成，以节点的形式进行开发，其根本目的是通过节点将一个复杂的机器人系统细分为多个可执行程序的最小单元。

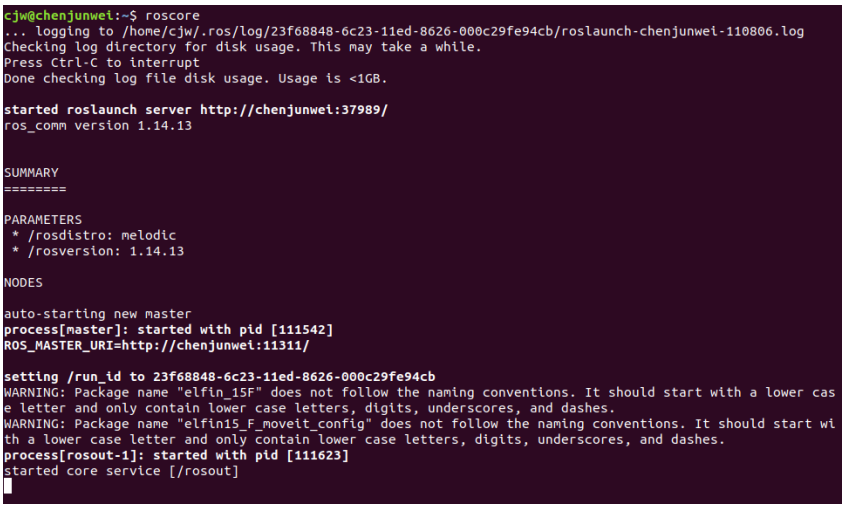
节点使基于 ROS 的系统在运行时更形象化，当多个节点同时进行时，不同节点的通信如下图所示



ROS Master:

由于机器人的各种元器件众多，功能庞大，在 ROS 系统中会产生大量的节点去负责不同的模块，而 ROS Master 是 ROS 提供的节点管理器，负责注册、管理 ROS 系统中的全部节点，在启动 ROS 节点之前首先需要启动 Master。

启动方式: roscore



ROS 节点(node)启动方式:

① rosrun: rosrun 包名 节点名称 参数名:=参数值(参数可有可无，具体看该节点需要)

② roslaunch: roslaunch 节点名称 launch 文件名.launch

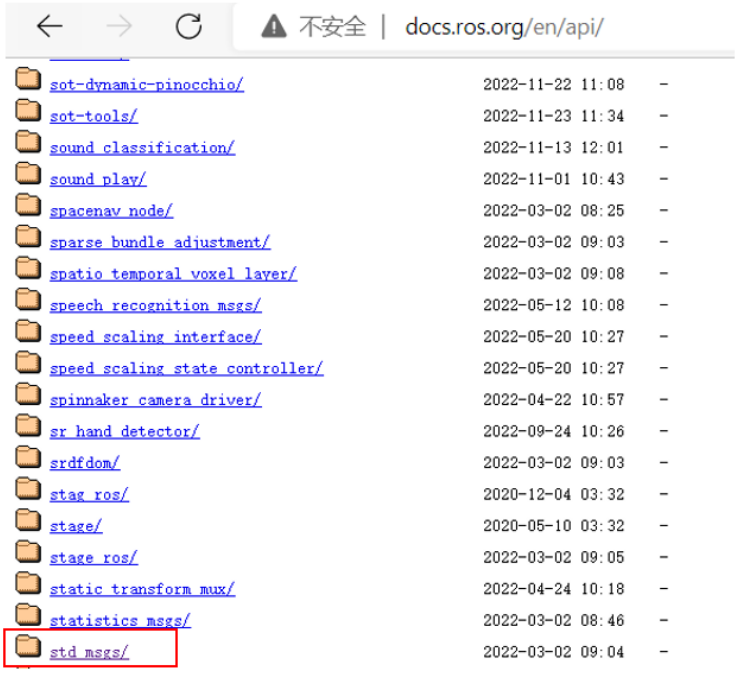
区别: rosrun 是运行一个单独节点的命令，如果要运行多个节点，则需要使用多次 rosrun 命令;而 roslaunch 可以用来启动定义在 launch 文件中的多个节点，通常的命名方案是以.launch 作为启动文件的后缀，启动文件是 xml 文件，一般把启动文件存储在取名为 launch 的目录中。

2.4.2.ROS 消息

节点之间通过传送消息进行通讯。每一个消息都是一种数据结构。

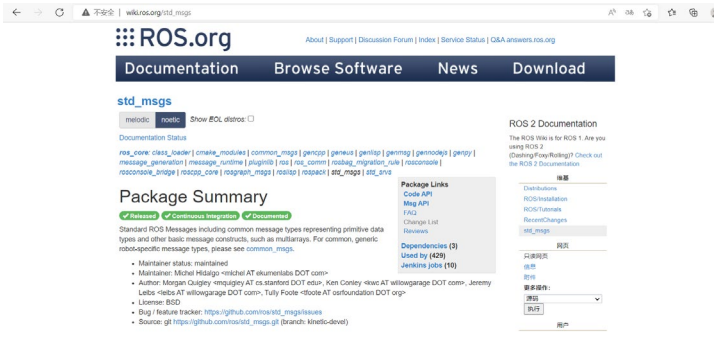
ROS 的消息支持标准的数据类型（整型、浮点型、布尔型等），还包括数组、结构体、自定义的数据类型等等。

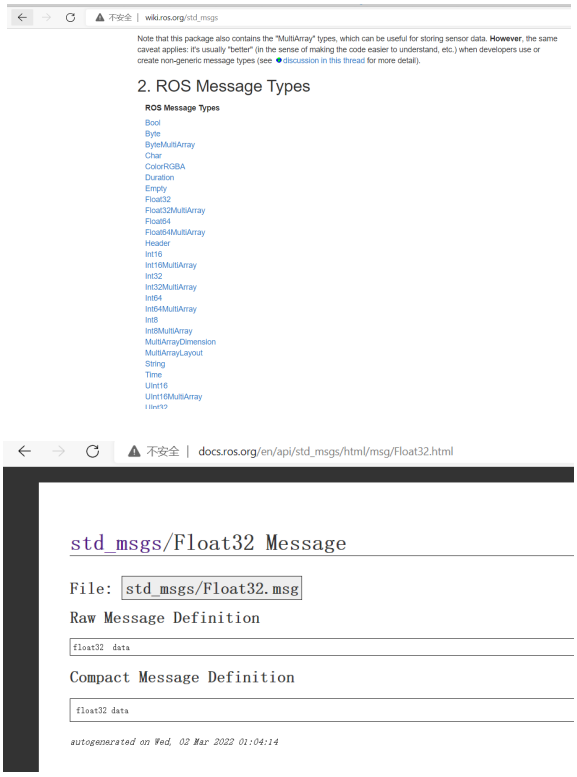
用户可通过 Index of /en/api (ros.org)查找不同的 ROS 的消息格式，例如：查找 std_msgs 中的 Float32.msg 的消息格式，可在该网址中搜索 std_msgs,打开后点击 html-std_msgs,然后会转到 wiki 中，往下拉找到 ROS Message Type,再点击 Float32 即可看到该消息的详细内容。


不安全 | docs.ros.org/en/api/
sot-dynamic-pinocchio/ 2022-11-22 11:08 -
sot-tools/ 2022-11-23 11:34 -
sound_classification/ 2022-11-13 12:01 -
sound_play/ 2022-11-01 10:43 -
spacenav_node/ 2022-03-02 08:25 -
sparse_bundle_adjustment/ 2022-03-02 09:03 -
spatio_temporal_voxel_layer/ 2022-03-02 09:08 -
speech_recognition_msgs/ 2022-05-12 10:08 -
speed_scaling_interface/ 2022-05-20 10:27 -
speed_scaling_state_controller/ 2022-05-20 10:27 -
spinnaker_camera_driver/ 2022-04-22 10:57 -
sr_hand_detector/ 2022-09-24 10:26 -
srdfdom/ 2022-03-02 09:03 -
stag_ros/ 2020-12-04 03:32 -
stage/ 2020-05-10 03:32 -
stage_ros/ 2022-03-02 09:05 -
static_transform_mux/ 2022-04-24 10:18 -
statistics_msgs/ 2022-03-02 08:46 -
std_msgs/ 2022-03-02 09:04 -


不安全 | docs.ros.org/en/api/std_msgs/
Index of /en/api/std_msgs
Name Last modified Size Description
Parent Directory -
html/ 2022-03-02 09:04 -
manifest.yaml 2022-03-02 09:04 10K
stamp 2022-03-02 09:04 0
Apache Server at docs.ros.org Port 80


不安全 | docs.ros.org/en/api/std_msgs/html/
Main Page Namespaces Classes Files
std_msgs Documentation
std_msgs
Author(s): Morgan Quigley , Ken Conley , Jeremy Leibs , Tully Foote
autogenerated on Wed Mar 2 2022 01:04:13


ROS.org
Documentation Browse Software News Download
std_msgs
Package Summary
ROS 2 Documentation



2.5.ROS 通信机制

* 该部分主要介绍 ROS 相关的入门基础，有需要的可以补充

注：本章节中部分的示例截图代码并不完全，需要的请到 ros_demo 包中进行查看。本章节中的所有示例均使用 rosrun 的方式启动节点。C++需要运行编译后的可执行文件，Python 可直接运行 python 文件。

2.5.1.话题通信(Topic)

2.5.1.1. 基本说明

话题通信是 ROS 中使用频率最高的一种通信模式，话题通信基于发布订阅的模式进行收发，即一个节点发布消息，一个节点订阅消息。通过话题通信可以实现不同节点之间的数据交互的通信模式，主要用于不断更新的、少逻辑处理的数据传输场景，例如传感器数据传输、摄像头等。

2.5.1.2. C++示例

发布者(Publisher)示例：

topic_pub_demo.cpp

```cpp
#include <ros/ros.h>
#include <std_msgs/String.h>
#include <sstream>
int main(int argc, char  *argv[])
{
    ros::init(argc,argv,"pub_demo_node");
    ros::NodeHandle nh;
    ros::Publisher pub = nh.advertise<std_msgs::String>("topic_demo",10);
    std_msgs::String msg;
    std::string msg_front = "Hello";
    int count = 0;
    ros::Rate r(1);
    ros::Duration(1.0).sleep();
    while (ros::ok())
    {
        std::stringstream ss;
        ss << msg_front << count;
        msg.data = ss.str();
        pub.publish(msg);
        ROS_INFO("publish msg:%s",msg.data.c_str());
        r.sleep();
        count++;
        ros::spinOnce();
    }
    return 0;
}

```

接收者(Subcriber)示例：

topic_sub_demo.cpp

```cpp
#include <ros/ros.h>
#include <std_msgs/String.h>

void doMsg(const std_msgs::String::ConstPtr& msg_p){
    ROS_INFO("Listen:%s",msg_p->data.c_str());
}
int main(int argc, char  *argv[])
{
    ros::init(argc,argv,"sub_demo_node");
    ros::NodeHandle nh;
    ros::Subscriber sub = nh.subscribe<std_msgs::String>("topic_demo",10,doMsg);

    ros::spin();

    return 0;
}

```

2.5.1.3. Python 示例

发布者(Publisher)示例：

topic_pub_demo_py.py

```python
#! /usr/bin/env python
import rospy
from std_msgs.msg import String

if __name__ == "__main__":
    rospy.init_node("pub_py_node")
    pub = rospy.Publisher("topic_py_demo",String,queue_size=10)
    msg = String()
    msg_front = "Hello"
    count = 0
    rate = rospy.Rate(1)
    rospy.Rate(1.0).sleep()
    while not rospy.is_shutdown():
        msg.data = msg_front + str(count)
        pub.publish(msg)
        rate.sleep()
        rospy.loginfo("publish msg:%s",msg.data)
        count += 1

```

接收者(Subcriber)示例：

topic_sub_demo_py.py

```python
#! /usr/bin/env python
import rospy
from std_msgs.msg import String

def doMsg(msg):
    rospy.loginfo("Listen:%s",msg.data)

if __name__ == "__main__":
    rospy.init_node("sub_py_node")
    sub = rospy.Subscriber("topic_py_demo",String,doMsg,queue_size=10)

    rospy.spin()

```

2.5.2.服务通信(Service)

2.5.2.1. 基本说明

服务通信在 ROS 也是一种比较频繁使用的一种通信模式，服务通信基于请求响应的模式，是一种应答机制。通过请求响应的方式可以实现不同节点之间的数据交互的通信模式，主要用于偶然的、实时性有要求、有一定逻辑处理需求的数据传输场景。

2.5.2.2. C++示例

服务端(Server)示例：

service_server_demo.cpp

```cpp
#include <ros/ros.h>
#include <ros_demo/service_demo.h>

bool doReq(ros_demo::service_demo::Request& req,
           ros_demo::service_demo::Response& resp){
    int num1 = req.num1;
    int num2 = req.num2;
    ROS_INFO("服务器接收到的请求数据为:num1 = %d, num2 = %d",num1, num2);
    if (num1 < 0 || num2 < 0)
    {
        ROS_ERROR("数据异常:数据不可以为负数");
        return false;
    }
    resp.sum = num1 + num2;
    return true;
}
int main(int argc, char *argv[])
{
    setlocale(LC_ALL,"");
    ros::init(argc,argv,"ros_service_demo_node");
    ros::NodeHandle nh;
    ros::ServiceServer server = nh.advertiseService("ros_service_demo",doReq);
    ROS_INFO("服务启动...");
    ros::spin();
    return 0;
}
```

客户端(Client)示例：

service_client_demo.cpp

ros_demo > src > service_client_demo.cpp > main(int, char * [])

```cpp
#include <ros/ros.h>
#include <ros_demo/service_demo.h>
int main(int argc, char *argv[])
{
    setlocale(LC_ALL,"");
    if (argc != 3)
    {
        ROS_ERROR("请输入两个整数");
    }
    ros::init(argc,argv,"ros_client_demo_node");
    ros::NodeHandle nh;
    ros::ServiceClient client = nh.serviceClient<ros_demo::service_demo>("ros_service_demo"
    ros::service::waitForService("ros_service_demo");
    ros_demo::service_demo ai;
    ai.request.num1 = atoi(argv[1]);
    ai.request.num2 = atoi(argv[2]);
    bool flag = client.call(ai);
    if (flag)
    {
        ROS_INFO("正常处理请求,响应结果:%d",ai.response.sum);
    }
    else
    {
        ROS_ERROR("请求处理失败....");
    }
    return 0;
}
```

2.5.2.3. Python 示例

服务端(Server)示例：

service_server_demo_py.py

ros_demo > scripts > service_server_demo_py.py > doReq

```python
#! /usr/bin/env python
# -*- coding: utf-8 -*-
import rospy
from ros_demo.srv import service_demo, service_demoRequest, service_demoResponse
def doReq(req):
    sum = req.num1 + req.num2
    rospy.loginfo("接受到客户端数据:num1 = %d, num2 = %d, 返回结果sum = %d",req.num1, req.num2,

    resp = service_demoResponse(sum)
    return resp


if __name__ == "__main__":
    rospy.init_node("ros_service_demo_py_node")
    server = rospy.Service("ros_service_demo",service_demo,doReq)
    rospy.loginfo("启动服务...")
    rospy.spin()
```

客户端(Client)示例：

service_client_demo_py.py

ros_demo > scripts > service_client_demo_py.py > ...

```python
#! /usr/bin/env python2
# -*- coding: utf-8 -*-
import rospy
from ros_demo.srv import service_demo, service_demoRequest, service_demoResponse
import sys

if __name__ == "__main__":
    rospy.init_node("ros_client_demo_py_node")

    if len(sys.argv) != 3:
        rospy.logerr("未输入正确参数")
        sys.exit(1)

    client = rospy.ServiceProxy("ros_service_demo",service_demo)
    client.wait_for_service()
    req = service_demoRequest()
    req.num1 = int(sys.argv[1])
    req.num2 = int(sys.argv[2])
    resp = client.call(req)
    rospy.loginfo("响应结果:%d",resp.sum)
```

2.5.3.动作通信(Action)

2.5.3.1. 基本说明

在 ROS 中提供了 actionlib 功能包集，用于实现动作通信。动作通信是类似服务通信的一种，其中也包含了请求和响应，但不同点在于在请求和响应过程中，动作服务端会连续的反馈当前任务的状态，动作客户端可以持续获取当前任务的状态反馈，同时可以取消当前任务。常用于耗时的请求响应场景，通过连续获取任务的状态反馈去判断是否需要终止或重新执行当前任务。

2.5.3.2. C++示例：

Action 服务端：

action_server_demo.cpp

ros_demo > src > action_server_demo.cpp > cb(const ros_demo::action_demoGoalConstPtr, Server *)

```cpp
#include <ros/ros.h>
#include <actionlib/server/simple_action_server.h>
#include <ros_demo/action_demoAction.h>

typedef actionlib::SimpleActionServer<ros_demo::action_demoAction> Server;

void cb(const ros_demo::action_demoGoalConstPtr goal,Server* server){
    int num = goal->num;
    ROS_INFO("目标值:%d",num);
    int result = 0;
    ros_demo::action_demoFeedback feedback;
    ros::Rate rate(10);
    for (int i = 1; i <= num; i++)
    {
        result += i;
        feedback.status = i / (double)num;
        server->publishFeedback(feedback);
        rate.sleep();
    }
    ros_demo::action_demoResult r;
    r.result = result;
    server->setSucceeded(r);
    ROS_INFO("结果:%d",r.result);
}

int main(int argc, char *argv[])
```

Action 客户端：

action_client_demo.cpp

ros_demo > src > action_client_demo.cpp > ...

```cpp
#include <ros/ros.h>
#include <actionlib/client/simple_action_client.h>
#include <ros_demo/action_demoAction.h>

typedef actionlib::SimpleActionClient<ros_demo::action_demoAction> Client;

void done_cb(const actionlib::SimpleClientGoalState &state, const ros_demo::action_demo
    if (state.state_ == state.SUCCEEDED)
    {
        ROS_INFO("结果:%d",result->result);
    } else {
        ROS_INFO("任务失败! ");
    }

}

void active_cb(){
    ROS_INFO("开启Action服务....");
}

void feedback_cb(const ros_demo::action_demoFeedbackConstPtr &feedback){
    ROS_INFO("当前进度:%.2f",feedback->status);
}

int main(int argc, char *argv[])
```

2.5.3.3. Python 示例：

Action 服务端：

```python
#! /usr/bin/env python
# -*- coding: utf-8 -*-

import rospy
import actionlib
from ros_demo.msg import *

class ActionServerDemo:
    def __init__(self):
        self.server = actionlib.SimpleActionServer("action_server_demo",action_demoAct
        self.server.start()
        rospy.loginfo("启动Action服务")

    def cb(self,goal):
        rospy.loginfo("服务端处理请求:")
        num = goal.num
        rate = rospy.Rate(10)
        sum = 0
        for i in range(1,num + 1):
            sum = sum + i
            feedBack = float(i*1.0/num)
            rospy.loginfo("当前进度:%.2f",feedBack)
            feedBack_obj = action_demoFeedback()
            feedBack_obj.status = feedBack
            self.server.publish_feedback(feedBack_obj)
            rate.sleep()
        result = action_demoActionResult()
```

Action 客户端：

```python
#! /usr/bin/env python
# -*- coding: utf-8 -*-

import rospy
import actionlib
from ros_demo.msg import *

def done_cb(state,result):
    if state == actionlib.GoalStatus.SUCCEEDED:
        rospy.loginfo("结果反馈:%d",result.result)

def active_cb():
    rospy.loginfo("服务已启动...")

def fb_cb(fb):
    rospy.loginfo("进度反馈:%.2f",fb.status)

if __name__ == "__main__":
    rospy.init_node("action_client_demo_py")
    client = actionlib.SimpleActionClient("action_server_demo",action_demoAction)
    client.wait_for_server()
    goal_obj = action_demoGoal()
    goal_obj.num = 10
    client.send_goal(goal_obj,done_cb,active_cb,fb_cb)
    rospy.spin()
```

2.5.4.常用命令

2.5.4.1. rosnode

用于获取节点相关的信息

rosnode list	列出所有活动的节点
rosnode ping	测试到节点的连接情况
rosnode info	打印节点信息
rosnode machine	列出指定设备上的节点

rosnode kill	杀死某个节点
rosnode cleanup	清楚不可连接的节点

2.5.4.2. rostopic

用于显示 ROS 主题的调试信息

rostopic list	列出所有活动的节点
rostopic bw	显示主题所用的带宽
rostopic info	显示主题相关信息
rostopic delay	显示带有 Header 的主题延迟
rostopic echo	打印主题信息到终端
rostopic find	根据类型查找主题
rostopic hz	显示主题的发布频率
rostopic pub	将数据发布到主题
rostopic type	打印主题类型

2.5.4.3. rosmsg

用于显示有关 ROS 消息类型的信息

rosmsg show	显示消息描述
rosmsg info	显示消息信息
rosmsg list	显示所有消息
rosmsg md5	显示 md5 加密后的信息
rosmsg package	显示某个功能包下的所有消息
rosmsg packages	列出包含消息的功能包

2.5.4.4. rosservice

用于列出和查询 ROS Service 的相关信息

rosservice args	打印服务参数
rosservice call	使用提供的参数调用服务
rosservice find	按照服务类型查找服务
rosservice info	打印有关服务的信息
rosservice list	列出所有活动的服务
rosservice type	打印服务类型
rosservice uri	打印服务的 ROSRPC uri

2.5.4.5. rossrv

用于 ROS 服务类型的相关信息，与 rosmsg 用法雷同

rossrv show	显示服务消息描述
rossrv info	显示服务消息的相关信息
rossrv list	显示所有服务信息
rossrv md5	显示 md5 加密后的服务消息
rossrv package	显示某个功能包下的所有服务消息
rossrv packages	列出包含服务消息的功能包

2.5.4.6. rosparam

用于使用 YAML 文件在参数服务器上获取和设置相关 ROS 参数

rosparam set	设置参数
rosparam get	获取参数
rosparam load	从外部文件加载参数
rosparam dump	将参数写出到外部文件
rosparam delete	删除参数
rosparam list	列出所有参数

第三章 Elfin-ROS 基本教程

注: 本部分基于 Melodic 版本进行介绍说明，对于其他版本可能存在些许不同，但改动不大，可参考功能包中的 readme 文件和 docs 文件夹中的说明文件。

3.1.环境配置

3.1.1.安装环境

3.1.1.1. 重要依赖包

sudo apt-get install ros-melodic-soem ros-melodic-gazebo-ros-control ros-melodic-ros-control ros-melodic-ros-controllers

3.1.1.2. 安装/升级 moveit

```
sudo apt-get update
sudo apt-get install ros-melodic-moveit-*
```

3.1.1.3. 安装 trac_ik 插件包

```
sudo apt-get install ros-melodic-trac-ik
```

3.1.1.4. 安装 elfin-ros 软件包

在安装 elfin-ros 软件包之前，需要先创建好工作空间，对于初学者可以参考本文档中的 2.3.2.1 章节，从创建工作空间到编译工作空间部分即可;对于有 ros 使用经验的用户，可自行下载到自己的工作空间中。

对于初学者，按照以下顺序进行安装:

```
cd ~/catkin_ws
git clone https://github.com/hans-robot/elfin_robot.git
cd ..
catkin_make
source devel/setup.bash
```

(注：本次安装默认为 melodic 分支，如需下载其他版本或其他分支请自行修改第二部分为所需的分支)

3.1.2.仿真环境测试

下面给出的是启动 Elfin5 的一系列命令。启动 Elfin3 和 Elfin10 的方法与之类似，只要在相应的地方替换掉相应的前缀即可。

启动 gazebo 仿真：

roslaunch elfin_gazebo elfin5_empty_world.launch

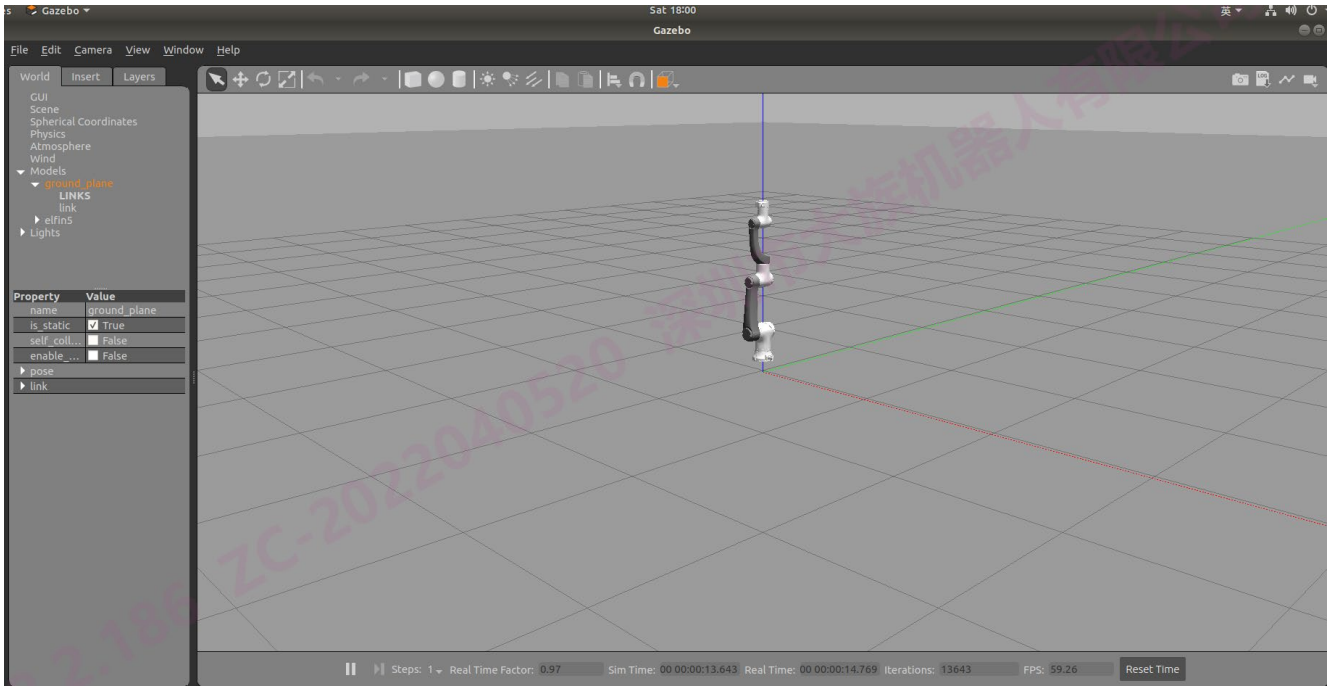
启动 moveit 和 rviz 界面
roslaunch elfin5_moveit_config moveit_planning_execution.launch
如果不想启动 rviz 界面，可使用以下命令:

roslaunch elfin5_moveit_config moveit_planning_execution.launch display:=false

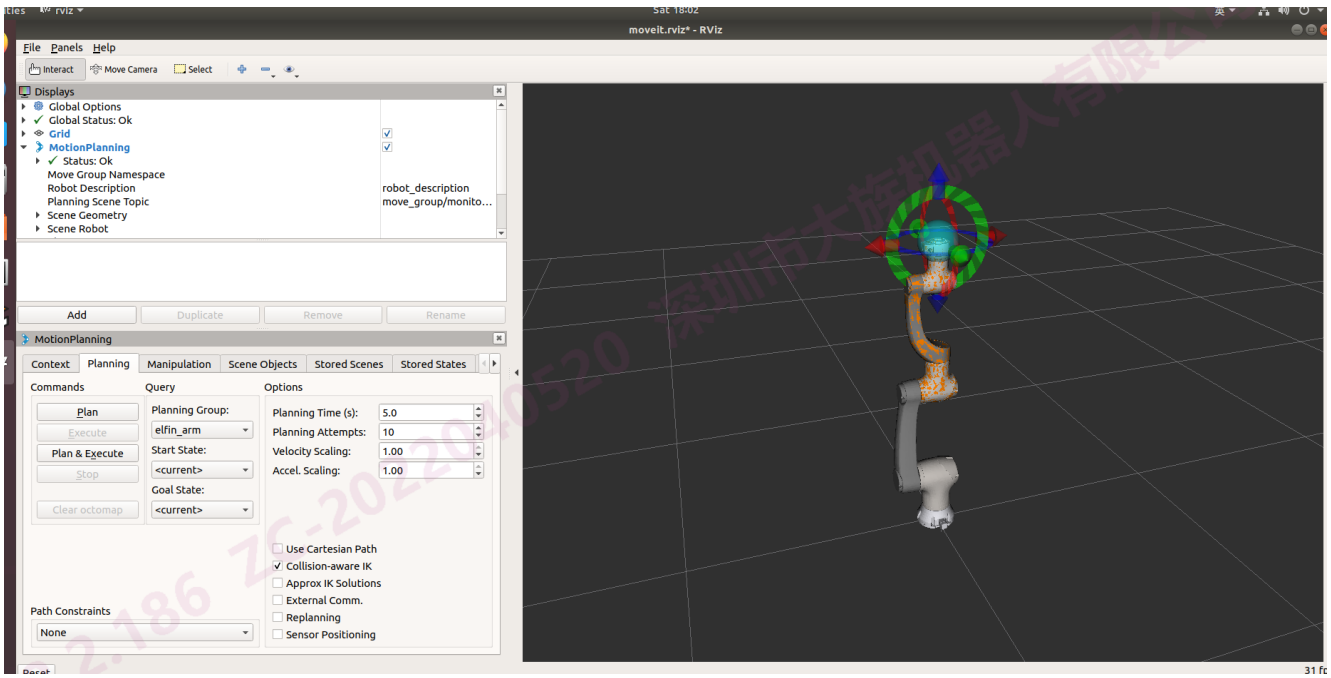
运行后台程序及 Elfin Control Panel 界面：

roslaunch elfin_basic_api elfin_basic_api.launch

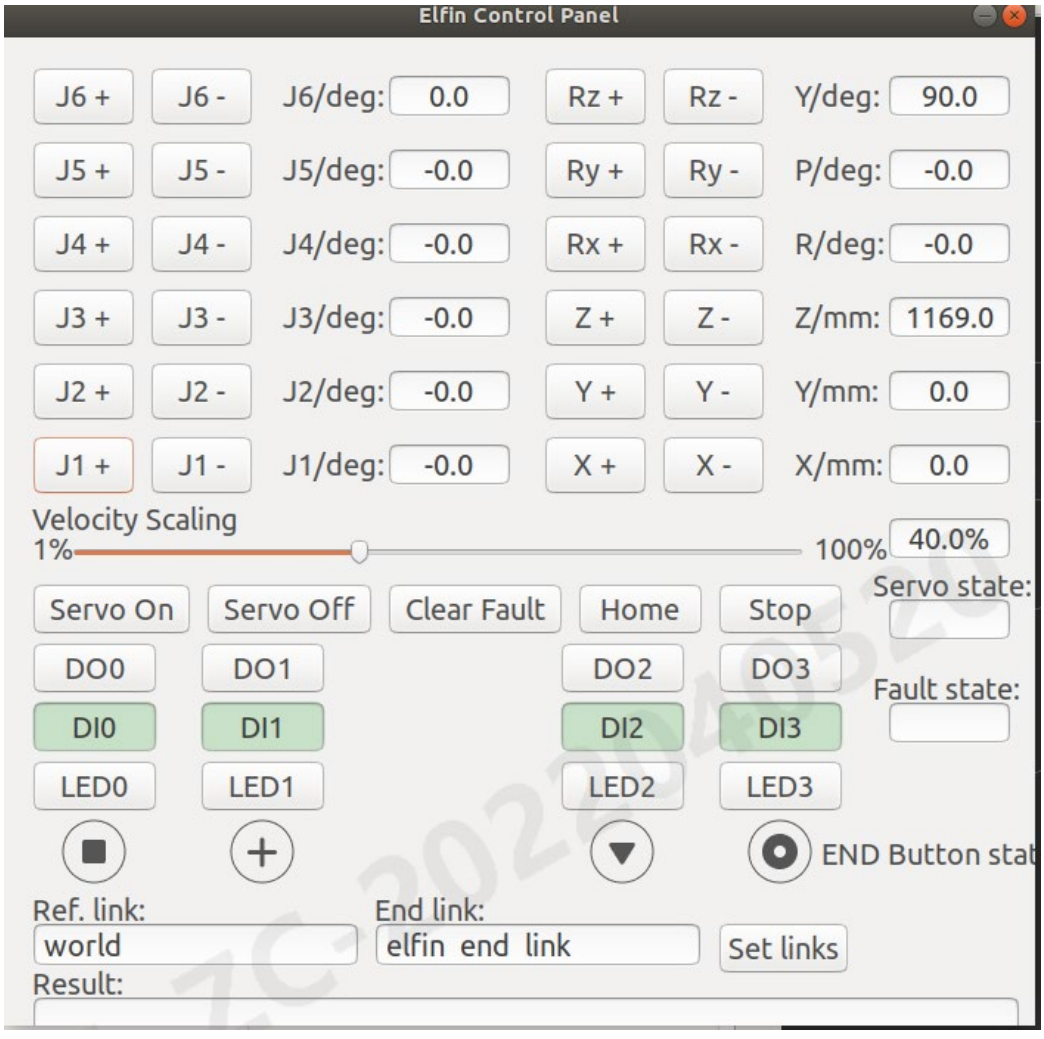
如果配置正确，将会看到以下几个界面：



Gazebo 仿真环境



Moveit&Rviz 界面



Elfin Control Panel 界面

3.2.通过 ROS 控制真实 Elfin 机器人

3.2.1.分支说明

目前在官网中共有 9 个分支，分别包括 indigo 版本、kinetic 版本、melodic 版本和 noetic 版本，其中 indigo 和 kinetic 已停止更新维护，详细见下边。

分支名	对应 Ubuntu 版本	状态	说明
master	Ubuntu14/16	已停止更新维护	主分支，为旧版本
indigo-devel	Ubuntu14	已停止更新维护	Indigo 版本
kinetic-devel	Ubuntu16	已停止更新维护	Kinetic 版本
melodic-devel	Ubuntu18	已停止更新维护	Melodic 版本
melodic-old-control-devel	Ubuntu18	已停止更新维护	包含旧版机器的控制器分支
noetic	Ubuntu20	维护中	Noetic 版本，485 末端
melodic_ethercat	Ubuntu18	维护中	Melodic 版本，总线末端
noetic_ethercat	Ubuntu20	维护中	Noetic 版本，总线末端
melodic	Ubuntu18	维护中	Melodic 版本，485 末端

注：目前默认分支为 Melodic，如需下载其他分支请使用该命令下载：git clone -b 分支名 https://github.com/hans-robot/elfin_robot.git

3.2.2.整体连接过程

整体连接过程包括两部分，分为硬件连接部分和软件配置部分。硬件连接部分为 Elfin 本体与安装好 ROS 的 PC 连接，软件配置部分为配置 ROS 包中的启动配置文件和连接 Elfin 机器人。

注：1. 在使用 ROS 连接前，最好将机器人回到零点，以便于后续判断配置是否正确。

2. 由于同一款机器人可能有多种不同高度的机型，用肉眼并无法很好分辨，请在回零后请记下当前示教器中点动页面的机器人高度，然后在连接上 ROS 后读取 Elfin Control Panel 界面中的高度，如果两者高度不匹配，请到：hans-robot/elfin_model: Store models of all hans-robot,including xacro files for ROS1 and ROS2 (github.com)（https://github.com/hans-robot/elfin_model）中按照说明重新配置机器人的 ROS 模型并重新运行。

3.2.3.硬件连接

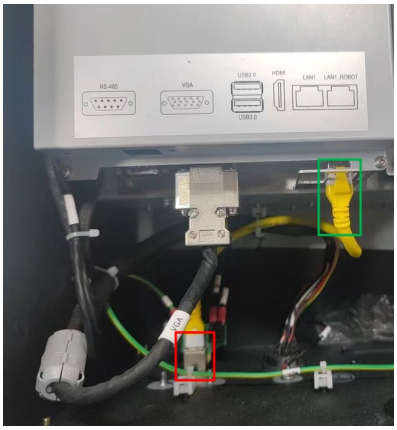
注:

① 在连接前请先确认使用的 PC 可连接使用的网口，不能使用 USB 转网口模块

② 在进行硬件连接前最好提前准备一个网口转接头(网口转网口)和一根网线



打开 Elfin 控制电箱后可以看到有一根网线，一端连接电箱中的工控机(如下图中绿色方框)一端连接机器人本体(如下图中红色方框)。

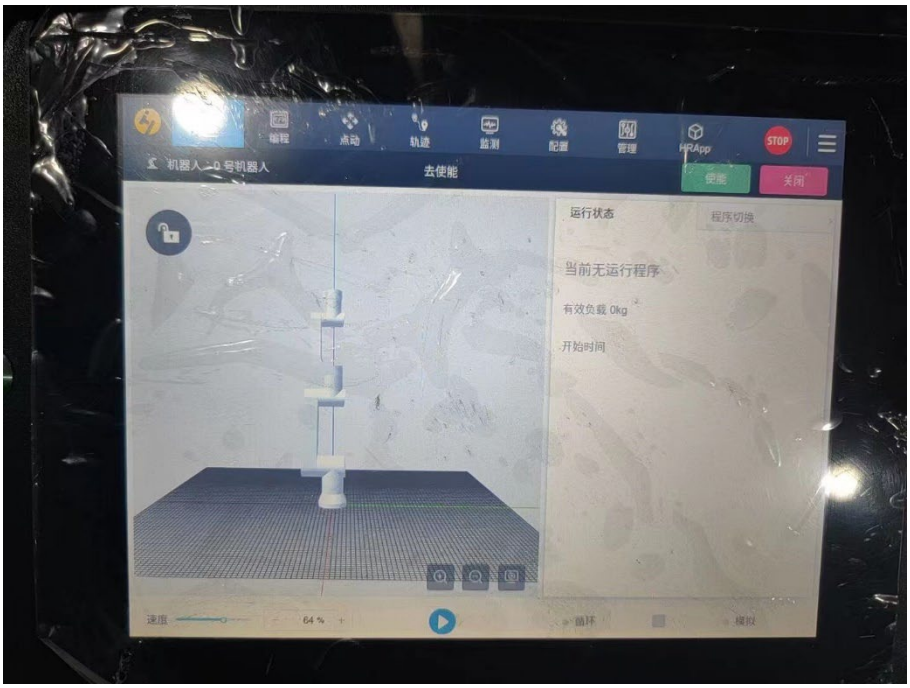


当没有提前准备好网口转接头时，需要拔出红色方框中的一端，并将提前准备好的网线接入，另一端网口接入安装好 ROS 的 PC 网口端。

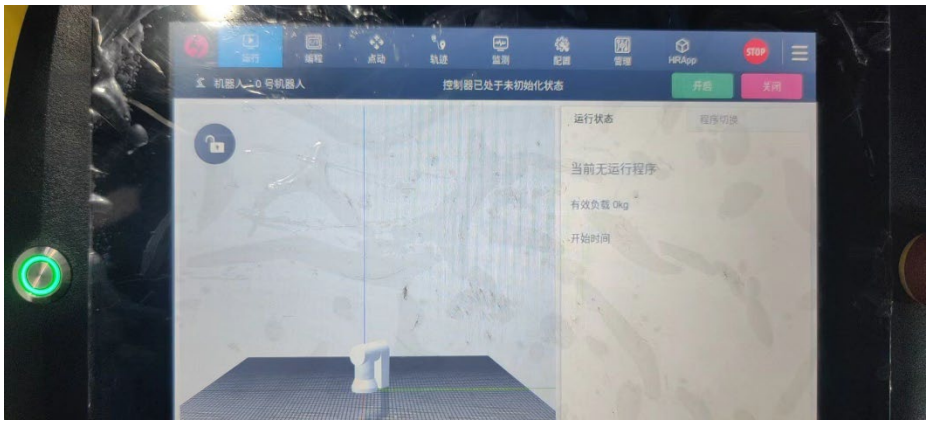
如果提前准备好网口转接头，推荐拔出绿色方框中的一端接入到网口转接头中的一端，并将提前准备好的网线一端接到网口转接头的另一端，网线的另一端接入安装 ROS 的 PC 网口端中。比较推荐使用网口转接头进行连接。

在完成接线后，可通过示教器开机，在开机完成进入控制界面后点击本体供电，等待供电完成后会发现

右上角机器人会一直处于初始化控制器状态，而并非在使用电箱控制时会跳转到去使能状态；



正常使用控制器时的状态



使用 ROS 连接时的状态

之后在 ROS 的 PC 中打开终端输入 ifconfig 查看当前网络设备状态，此时会多出一个网络设备信息，需要记录下该设备信息中的网卡名称，用于后续的软件配置。

连接过程可以查看以下链接中的操作视频：

链接: https://pan.baidu.com/s/1gsnbhXR8o-QY9feg8tVKeA 提取码: rfbt

3.2.4.软件配置

在完成硬件连接后需要进一步对 ROS 包中的配置文件进行配置，主要是配置 elfin_robot_bringup/config/文件夹下的 elfin_drivers.yaml 文件。

Open ▾ elfin_drivers.yaml ~/catkin_ws/src/elfin_robot/elfin_robot_bringup/config Save

```
## elfin_ethernet_name: network interface card name
## An Elfin robot has 3 modules and 6 joints.
## All the parameters about joints should be in order of [J2, J1, J3, J4, J5, J6]
## reduction_ratios: The reduction ratios of the joints.
##                   Please refer to the paper attached with the robot.
## count_zeros: The encoder counts of the joints when they are in home position.
##              Please refer to the paper attached with the robot.
## axis_position_factors: The encoder resolution values. They are all 131072 in general.
## axis_torque_factors: They depend on the robot type.
##                      Elfin3: [5251.283, 5251.283, 8533.125, 8533.125, 15975.05, 15975.05]
##                      Elfin5: [2536.224, 2536.224, 5251.283, 5251.283, 15975.05, 15975.05]
##                      Elfin10: [1365.332, 1365.332, 2536.224, 2536.224, 8533.125, 8533.125]

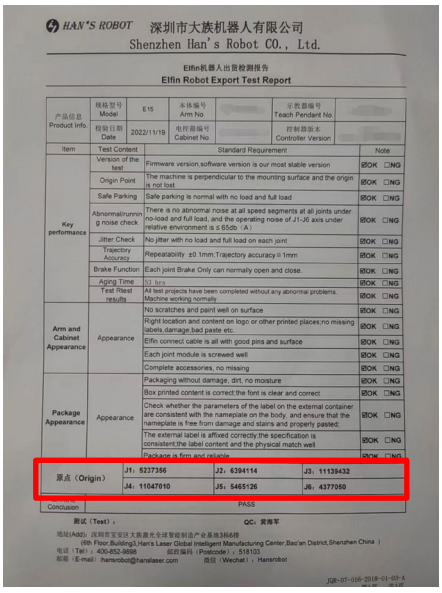
## example for elfin_robot_bringup/config/elfin_drivers.yaml
elfin_ethernet_name: enp0s3
elfin:
    slave_no: [1, 2, 3]
    joint_names: [elfin_joint2, elfin_joint1, elfin_joint3, elfin_joint4, elfin_joint5,
elfin_joint6]
    reduction_ratios: [101, 101, 101, 101, 101, 101]
    count_zeros: [8907490, 9037465, 5336080, 5456119, 4246746, 1592228]
    axis_position_factors: [131072, 131072, 131072, 131072, 131072, 131072]
    axis_torque_factors: [5251.283, 5251.283, 8533.125, 8533.125, 15975.05, 15975.05]
```

该配置文件需要修改的主要有四个部分：

第一步：修改 elfin_ethernet_name，将原有包中的 eth0 替换为通过 ifconfig 查看到的网卡名称，即上一步中所描述的网卡名。

第二步：修改 reduction_ratios(减速比)，原有包中的六个关节减速比默认为 101，对于 3KG、5KG 机型不需要修改，10KG 机型需要修改为 [121,121,101,101,101,101],15KG 机型需要修改为 [121,121,121,121,101,101]，对于其他机型请联系技术人员获取。

第三步：修改 count_zeros(原点值)，此处需要按照出货单中原点一栏的值进行修改，需要注意的是这里的顺序为 J2、J1、J3、J4、J5、J6，别把 J2 与 J1 原点值写反。



HAN'S ROBOT 深圳市大族机器人有限公司
Shenzhen Han's Robot CO., Ltd.

Elfin机器人出货检测报告
Elfin Robot Export Test Report

产品信息 Product Info.	规格型号 Model	E15	本体编号 Arm No.		示教器编号 Teach Pendant No.	
	检验日期 Date	2022/11/19	电控箱编号 Cabinet No.		控制器版本 Controller Version	

Item	Test Content	Standard Requirement	Note
Key performance	Version of the test	Firmware version,software version is our most stable version	☑OK ☐NG
	Origin Point	The machine is perpendicular to the mounting surface and the origin is not lost	☑OK ☐NG
	Safe Parking	Safe parking is normal with no load and full load	☑OK ☐NG
	Abnormal/running noise check	There is no abnormal noise at all speed segments at all joints under no-load and full load, and the operating noise of J1-J6 axis under relative environment is ≤ 65db（A）	☑OK ☐NG
	Jitter Check	No jitter with no load and full load on each joint	☑OK ☐NG
	Trajectory Accuracy	Repeatability ±0.1mm;Trajectory accuracy≤ 1mm	☑OK ☐NG
	Brake Function	Each joint Brake Only can normally open and close.	☑OK ☐NG
	Aging Time	53 hrs	☑OK ☐NG
	Test Rlest results	All test projects have been completed without any abnormal problems. Machine working normally	☑OK ☐NG
Arm and Cabinet Appearance	Appearance	No scratches and paint well on surface	☑OK ☐NG
		Right location and content on logo or other printed places;no missing labels,damage,bad paste etc.	☑OK ☐NG
		Elfin connect cable is all with good pins and surface	☑OK ☐NG
		Each joint module is screwed well	☑OK ☐NG
		Complete accessories, no missing	☑OK ☐NG
Package Appearance	Appearance	Packaging without damage, dirt, no moisture	☑OK ☐NG
		Box printed content is correct,the font is clear and correct	☑OK ☐NG
		Check whether the parameters of the label on the external container are consistent with the nameplate on the body, and ensure that the nameplate is free from damage and stains and properly pasted;	☑OK ☐NG
		The external label is affixed correctly,the specification is consistent,the label content and the physical match well	☑OK ☐NG
		Package is firm and reliable	☑OK ☐NG
原点（Origin）	J1：5237356	J2：6394114	J3：11139432
	J4：11047010	J5：5465126	J6：4377050
Conclusion	PASS		

测试（Test）： QC：黄海军

地址(Add)：深圳市宝安区大浪激光全球智能制造产业基地3栋6楼
(6th Floor,Building3,Han's Laser Global Intelligent Manufacturing Center,Bao'an District,Shenzhen China)
电话（Tel）：400-852-9898 邮政编码（Postcode）：518103
邮箱（E-mail）：hansrobot@hanslaser.com 微信（Wechat）：Hansrobot

JQR-07-016-2018-01-02-A

第四步：axis_position_factors 为编码器分辨率，基本机型默认为 131072;axis_torque_factors 为关节力矩转换常数，此处的值根据机型不同修改，对于其他机型请联系技术人员获取。

机型	关节力矩转换常数(J1、J2、J3、J4、J5、J6)

3KG	4552.118	4552.118	8183.818	8183.818	20480	20480
5KG	2340.838	2340.838	4552.118	4552.118	20480	20480
10KG	1241.026	1241.026	2202.685	2202.685	7884.808	7884.808
15KG	1023.976	1023.976	2340.838	2340.838	8183.818	8183.818

3.2.5.连接机器人

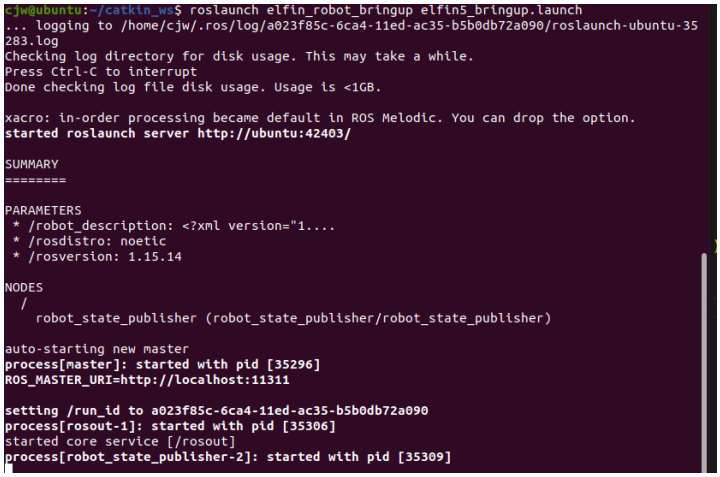
在完成硬件连接和软件配置后，连接机器人,以下每个步骤都需要开启一个新终端运行对应指令。

① 加载机器人模型：

roslaunch elfin_robot_bringup elfin5_bringup.launch

(此处加载的为 Elfin5 机器人，若使用其他机型请自行修改命令，如使用 Elfin3 机器人即修改为

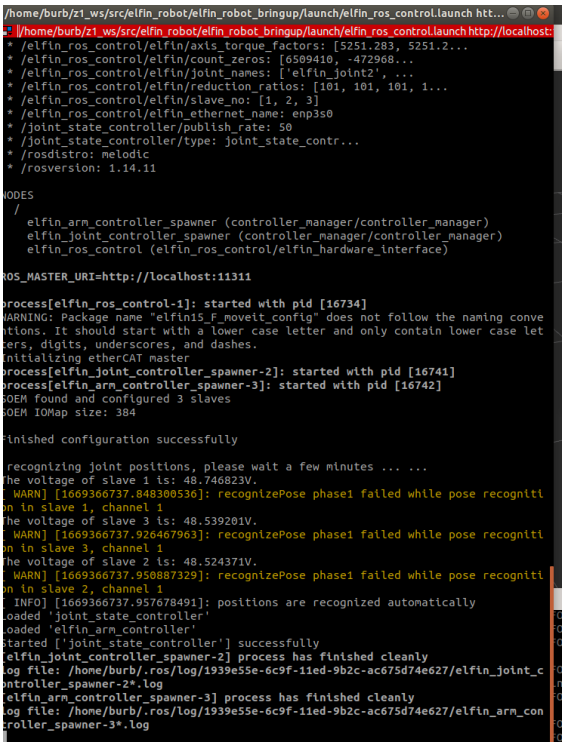
Roslaunch elfin_robot_bringup elfin3_bringup.launch 即可)



② 启动 Elfin 硬件，由于 ROS 为非实时系统，如果对实时性有要求需要自行安装实时内核补丁，具体可参考 realtime:documentation:howto:applications:preemptrt_setup [Wiki] (linuxfoundation.org)。

修改进程优先级：sudo chrt 10 bash

启动硬件连接：roslaunch elfin_robot_bringup elfin_ros_control.launch

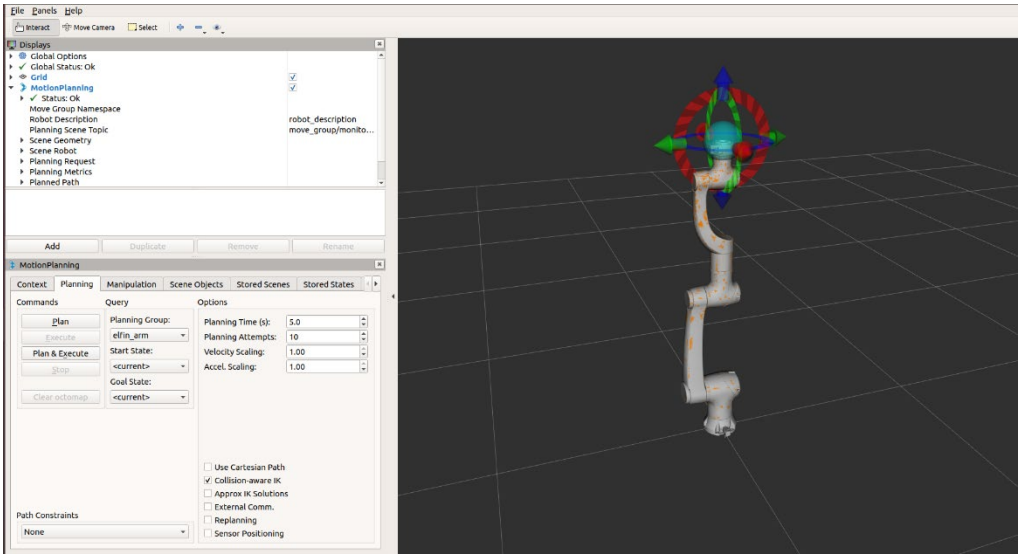


③ 启动 Moveit 和 Rviz 界面：

roslaunch elfin5_moveit_config moveit_planning_execution.launch

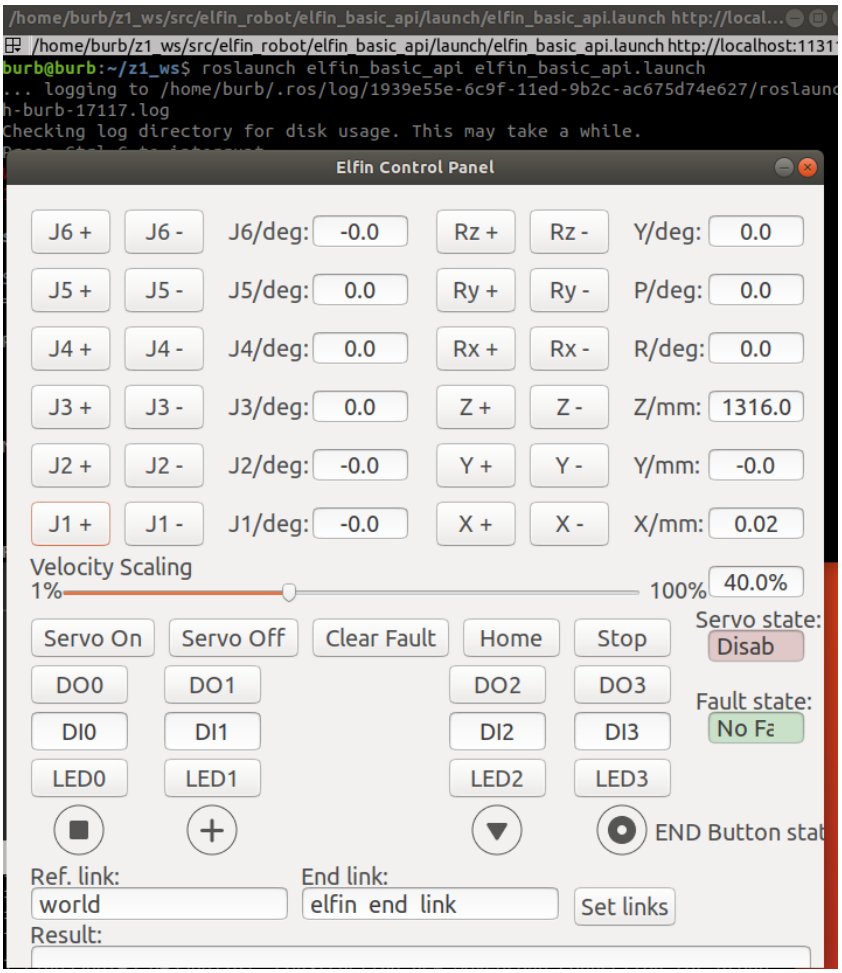
不需要 Rviz 界面可启动：roslaunch elfin5_moveit_config moveit_planning_execution.launch display:=false

(此处加载的为 Elfin5 机器人，若使用其他机型请自行修改命令，如使用 Elfin3 机器人即修改为 roslaunch elfin3_moveit_config moveit_planning_execution.launch 即可)



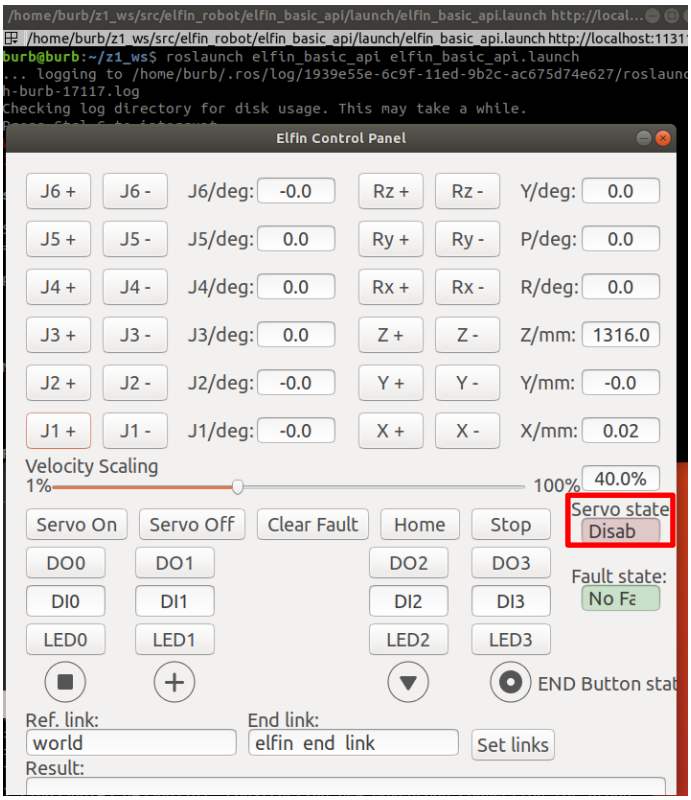
④ 运行后台程序及 Elfin Control Panel 界面：

roslaunch elfin_basic_api elfin_basic_api.launch

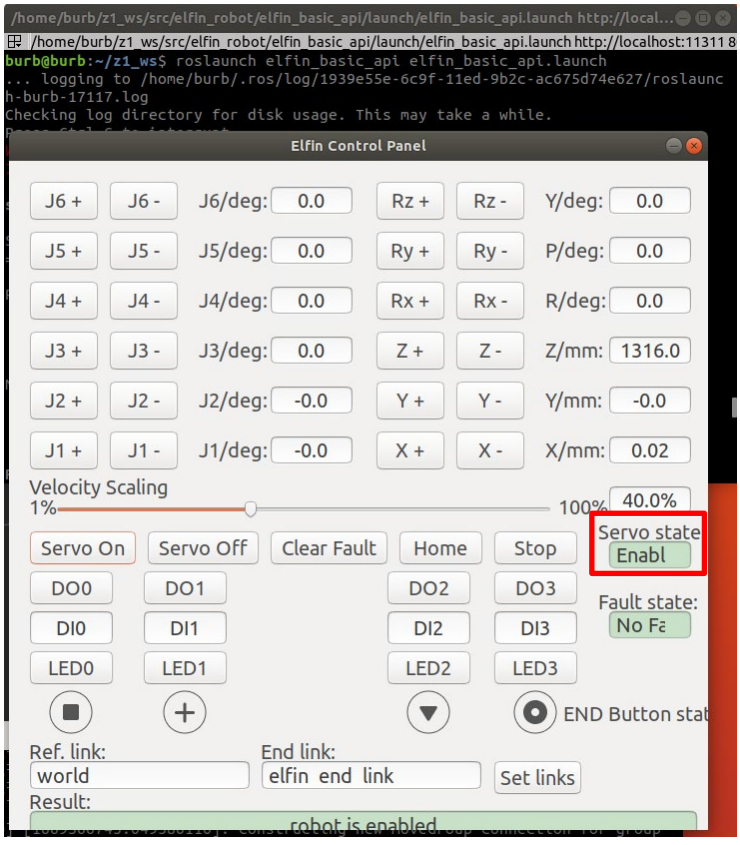


⑤ 开启使能：

通过点击 Elfin Control Panel 界面上的 Servo On 按钮对机器人使能，使能后界面会显示使能。



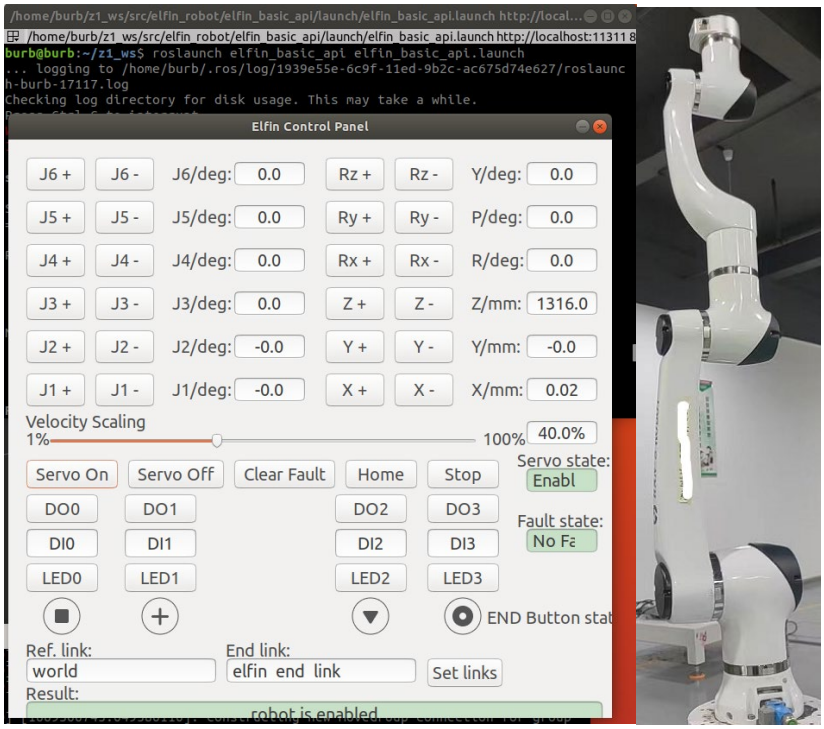
未使能



已使能

⑥ 查看界面关节值与实际关节值是否相等

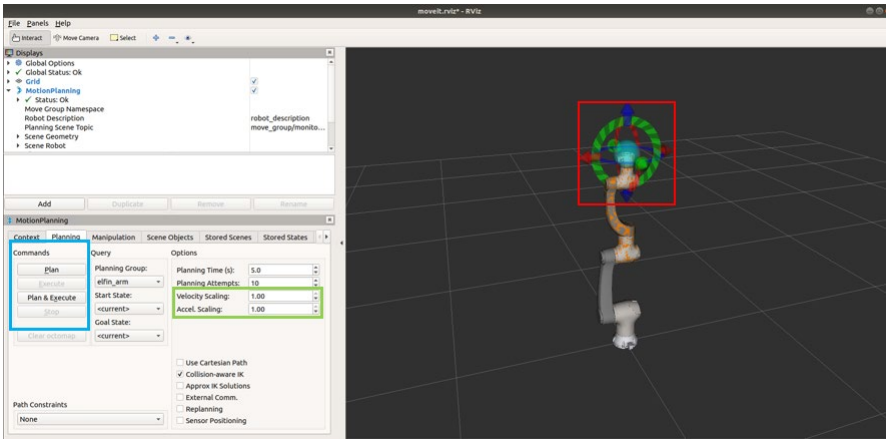
在机器人原点连接 ROS，配置正确的话在连接 ROS 后打开界面可看到关节值为 0 度,与实际机器人相同



3.2.6.基本控制使用

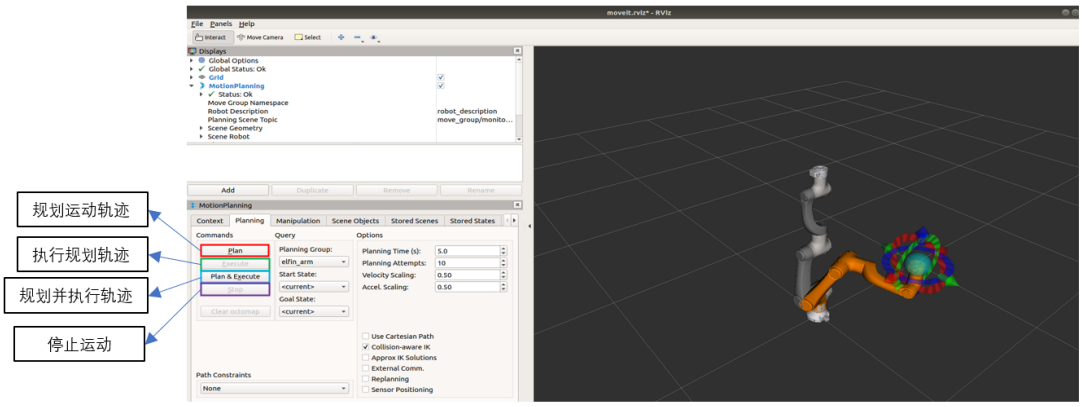
在完成连接机器人后，将会得到两个界面，一个是 Moveit-Rivz 界面一个是 Elfin Control Panel 界面，可通过 Moveit-Rviz 界面和 Elfin Control Panel 界面控制机器人进行基本的运动。

Moveit 控制：如下图所示，在 Rviz 界面中可拖动右侧框中的三维球移动模型的目标姿态，在拖动到预期位姿后可通过左侧蓝色框中的按钮进行规划并执行运动。注意在连接真实 Elfin 机器人时需要将绿色框中的速度比和加速度比进行调节，建议将速度比和加速度比修改为 0.5。

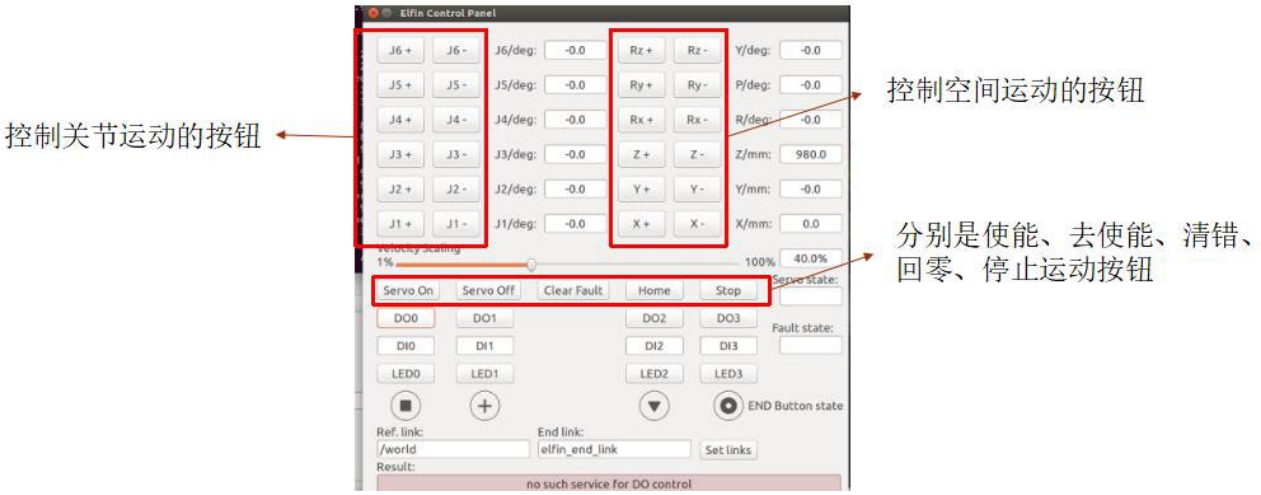


拖动三维球到一个期望姿态后，将速度比和加速度比修改为 0.5，点击左侧的规划运动轨迹按钮即可看到机器人从开始位置规划出一条轨迹运动到目标位置。

点击执行规划轨迹按钮即可看到模拟机器人与真实机器人开始运动到目标位置，点击停止运动即可在运动过程中停止机器人运动。规划并执行轨迹是规划运动轨迹和执行规划轨迹的集合，点击该按钮会马上进行轨迹规划并开始运动。



Elfin Control Panel 界面控制：如下图所示，在 Elfin Control Panel 界面中可通过点击控制关节按钮或控制空间运动按钮控制机器人运动



3.3.基本接口介绍

Elfin-ROS 接口分为话题接口和服务接口两部分，其中包括了运动控制接口、IO 接口、读取/设置相关参数/状态接口和硬件相关接口。

3.3.1.话题接口

话题接收接口：用户通过话题发布者形式向话题接收接口发送对应的话题信息去使用。



话题名	消息类型	接口所属	接口说明
elfin_basic_api/joint_goal	sensor_msgs/JointState	运动控制接口	令机械臂规划到达指定臂型的路径并执行此路径
elfin_basic_api/cart_goal	geometry_msgs/PoseStamped	运动控制接口	令机械臂规划到达指定空间位置的路径并执行此路径
elfin_basic_api/cart_path_goal	geometry_msgs/PoseArray	运动控制接口	令机械臂规划一条路径，以直线方式依次经过各个指定的点，并执行此路径
elfin_arm_controller/command	trajectory_msgs/JointTrajectory	运动控制接口	本消息内容为一条轨迹，可令机械臂沿着这条轨迹运动

话题发布接口：用户通过话题接收者的形式接收从话题发布者中发布出来的相关信息。



话题名	消息类型	接口所属	接口说明
elfin_arm_controller/state	control_msgs/JointTrajectoryControllerState	读取相关状态接口	反映机械臂各个关节的状态
elfin_ros_control/elfin/enable_state	std_msgs/Bool	读取相关状态接口	反映此时 Elfin 机械臂上电机的使能状态
elfin_ros_control/elfin/fault_state	std_msgs/Bool	读取相关状态接口	反映此时 Elfin 机械臂上电机的报错状态
elfin_basic_api/parameter_updates	dynamic_reconfigure/Config	读取相关参数接口	反映 elfin_basic_api 相关的动态参数

elfin_basic_api/reference_link_name	std_msgs/String	读取相关参数接口	elfin_basic_api 节点计算时所使用的参考基坐标系的名字
elfin_basic_api/end_link_name	std_msgs/String	读取相关参数接口	elfin_basic_api 节点计算时所使用的末端坐标系的名字

3.3.2.服务接口

服务接口需要用户以客户端(Client)形式实现，向服务端(Server)接口发送 call 指令调用，并自行接收服务端的返回结果去开发使用。

服务名	服务类型	接口所属	接口说明
elfin_basic_api/get_reference_link	std_srvs/SetBool	读取相关参数接口	呼叫本服务得到的反馈信息中会包含 elfin_basic_api 规划时的参考基坐标系名
elfin_basic_api/get_end_link	std_srvs/SetBool	读取相关参数接口	呼叫本服务得到的反馈信息中会包含 elfin_basic_api 规划时的末端坐标系名
elfin_basic_api/stop_teleop	std_srvs/SetBool	运动控制接口	呼叫本服务会令机械臂停止运动
elfin_basic_api/set_parameters	dynamic_reconfigure/Reconfigure	设置相关参数接口	设置 elfin_basic_api 相关的动态参数，例如: velocity scaling
elfin_ros_control/elfin/get_txpdo	std_srvs/SetBool	硬件相关接口	呼叫本服务得到的反馈信息中会包含机械臂上从站的 txpdo 信息
elfin_ros_control/elfin/get_rxpdo	std_srvs/SetBool	硬件相关接口	呼叫本服务得到的反馈信息中会包含机械臂上从站的 rxpdo 信息
elfin_ros_control/elfin/get_current_position	std_srvs/SetBool	硬件相关接口	呼叫本服务得到的反馈信息中会包含机械臂各轴的位置的编码器值
elfin_ros_control/elfin/recognize_position	std_srvs/SetBool	运动控制接口	呼叫本服务可使机械臂进行姿态识别
elfin_ros_control/elfin/io_port1/write_do	elfin_robot_msgs/ElfinIODWrite	IO 接口	向 DO 中写入内容
elfin_ros_control/elfin/io_port1/read_di	elfin_robot_msgs/ElfinIODRead	IO 接口	从 DI 中读取内容
elfin_ros_control/elfin/io_port1/get_txpdo	std_srvs/SetBool	硬件相关接口	呼叫本服务得到的反馈信息中会包含 IO 从站的 txpdo 信息
elfin_ros_control/elfin/io_port1/get_rxpdo	std_srvs/SetBool	硬件相关接	呼叫本服务得到的反馈信息中会

		口	包含 IO 从站的 rxpdo 信息
elfin_basic_api/enable_robot	std_srvs/SetBool	运动控控制接口	呼叫本服务可使机械臂使能
elfin_ros_control/elfin/enable_robot	std_srvs/SetBool	运动控控制接口	呼叫本服务可使机械臂使能。
elfin_basic_api/disable_robot	std_srvs/SetBool	运动控控制接口	呼叫本服务可使机械臂去使能
elfin_ros_control/elfin/disable_robot	std_srvs/SetBool	运动控控制接口	呼叫本服务可使机械臂去使能
elfin_ros_control/elfin/clear_fault	std_srvs/SetBool	硬件相关接口	呼叫本服务可使机械臂清除报错状态
elfin_basic_api/set_reference_link	elfin_robot_msgs/SetString	设置相关参数接口	设置 elfin_basic_api 节点计算时所使用的参考基坐标系
elfin_basic_api/set_end_link	elfin_robot_msgs/SetString	设置相关参数接口	设置 elfin_basic_api 节点计算时所使用的末端坐标系
elfin_basic_api/home_teleop	std_srvs/SetBool	运动控控制接口	呼叫本服务后，机械臂会一直运动，直到回到零位置或用户调用 elfin_basic_api/stop_teleop

3.4.常用接口使用&Demo

本章节仅对一些常用的控制接口、读取和设置参数/状态接口和 IO 接口进行详细描述并提供对应的 C++和 Python 示例，对于其他的接口可参考示例中的代码进行实现。

注：本章节中部分的示例截图代码并不完全，需要的请到 elfin_ros_demo 包中进行查看。本章节中的所有示例均使用 rosrun 的方式启动节点。C++需要运行编译后的可执行文件，Pyhon 可直接运行 python 文件。

3.4.1.控制接口

3.4.1.1. elfin_basic_api/joint_goal

说明：向机械臂发送关节目标值，消息类型为 sensor_msgs::JointState，用户需要实现一个话题发布者向 elfin_basic_api/joint_goal 发送期望的目标关节值，以弧度为单位。

C++示例：

```cpp
/*
Elfin-ROS控制接口示例
elfin_basic_api/joint_goal
*/

#include <ros/ros.h>
#include <sensor_msgs/JointState.h>

int main(int argc, char  *argv[])
{
    ros::init(argc,argv,"joint_goal_demo_node");
    ros::NodeHandle nh;
    ros::Publisher pub = nh.advertise<sensor_msgs::JointState>("elfin_basic_api/joi

    sensor_msgs::JointState pub_joint_goal;
    pub_joint_goal.name={"elfin_joint1", "elfin_joint2", "elfin_joint3",
                         "elfin_joint4", "elfin_joint5", "elfin_joint6"};

    pub_joint_goal.position={1.57,1.57,1.57,1.57,1.57,1.57};

    pub_joint_goal.header.stamp=ros::Time::now();


    ros::Rate r(1);
    ros::Duration(1.0).sleep();
    pub.publish(pub_joint_goal);
```

Python 示例：

```python
#!/usr/bin/env python2
# -*- coding: utf-8 -*-
"""
Elfin-ROS控制接口示例
elfin_basic_api/joint_goal
"""

import rospy
from sensor_msgs.msg import JointState

class Joint_Goal_Demo(object):
    def __init__(self):
        self.joints_pub=rospy.Publisher('elfin_basic_api/joint_goal', JointState

    def pub_joint_goal(self):
        js=JointState()
        js.name=['elfin_joint1', 'elfin_joint2', 'elfin_joint3',
                 'elfin_joint4', 'elfin_joint5', 'elfin_joint6']
        js.position=[1.57,1.57,1.57,1.57,1.57,1.57]
        js.header.stamp=rospy.get_rostime()
        self.joints_pub.publish(js)

if __name__=='__main__':
    rospy.init_node('cmd_pub', anonymous=True)
    test=Joint_Goal_Demo()
    rospy.sleep(1)
    test.pub_joint_goal()
```

3.4.1.2. elfin_basic_api/cart_goal

说明：向机械臂发送空间目标值，消息类型为 geometry_msgs/PoseStamped，用户需要实现一个话题发布者向 elfin_basic_api/cart_goal 发送期望的目前空间值，空间位置以米为单位。

C++示例：

```cpp
/*
Elfin-ROS控制接口示例
elfin_basic_api/cart_goal
*/

#include <ros/ros.h>
#include <geometry_msgs/PoseStamped.h>

int main(int argc, char  *argv[])
{
    ros::init(argc,argv,"cart_goal_demo_node");
    ros::NodeHandle nh;
    ros::Publisher pub = nh.advertise<geometry_msgs::PoseStamped>("elfin_basic_api/c

    geometry_msgs::PoseStamped pub_cart_goal;
    pub_cart_goal.header.stamp = ros::Time::now();
    pub_cart_goal.header.frame_id = "elfin_base_link";
    pub_cart_goal.pose.position.x = 0.42;
    pub_cart_goal.pose.position.y = 0.0;
    pub_cart_goal.pose.position.z = 0.445;
    pub_cart_goal.pose.orientation.x = 0;
    pub_cart_goal.pose.orientation.y = 1;
    pub_cart_goal.pose.orientation.z = 0;
    pub_cart_goal.pose.orientation.w = 0;

    ros::Rate r(1);
    ros::Duration(1.0).sleep();
```

Python 示例：

cart_goal_demo.py

```
#!/usr/bin/env python2
# -*- coding: utf-8 -*-
"""
Elfin-ROS控制接口示例
elfin_basic_api/cart_goal
"""

import rospy
from geometry_msgs.msg import PoseStamped

class Cart_Goal_Demo(object):
    def __init__(self):
        self.cart_pub=rospy.Publisher('elfin_basic_api/cart_goal', PoseStamped, queue_siz

    def pub_cart_goal(self):
        ps=PoseStamped()
        ps.header.stamp=rospy.get_rostime()
        ps.header.frame_id='elfin_base_link'
        ps.pose.position.x=0.420
        ps.pose.position.y=0.0
        ps.pose.position.z=0.445
        ps.pose.orientation.x=0
        ps.pose.orientation.y=1
        ps.pose.orientation.z=0
        ps.pose.orientation.w=0
        self.cart_pub.publish(ps)

```

3.4.1.3. elfin_basic_api/cart_path_goal

说明：向机械臂发送一组空间目标值，机械臂以直线运动的方式依次经过每个点，消息类型为 geometry_msgs/PoseArray，用户需要实现一个话题发布者向 elfin_basic_api/cart_path_goal 发送期望的目前空间组值，空间位置以米为单位

C++示例：

cart_path_goal_demo.cpp

```
*
lfin-ROS控制接口示例
lfin_basic_api/cart_path_goal
/

include <ros/ros.h>
include <geometry_msgs/PoseArray.h>
include <geometry_msgs/Pose.h>

nt main(int argc, char  *argv[])

   ros::init(argc,argv,"cart_path_goal_demo_node");
   ros::NodeHandle nh;
   ros::Publisher pub = nh.advertise<geometry_msgs::PoseArray>("elfin_basic_api

   geometry_msgs::PoseArray pub_cart_path_goal;
   geometry_msgs::Pose cart_path_goal;
   geometry_msgs::Pose cart_path_goal1;

   pub_cart_path_goal.header.stamp = ros::Time::now();
   pub_cart_path_goal.header.frame_id = "elfin_base_link";

   cart_path_goal.position.x=0.420;
   cart_path_goal.position.y=0.0;
   cart_path_goal.position.z=0.445;
   cart_path_goal.orientation.x=0;
```

Python 示例：

cart_path_goal_demo.py

```
#!/usr/bin/env python2
# -*- coding: utf-8 -*-
"""
Elfin-ROS控制接口示例
elfin_basic_api/cart_path_goal
"""

import rospy
from geometry_msgs.msg import PoseArray, Pose

class Cart_Path_Goal_Demo(object):
    def __init__(self):
        self.cart_path_pub=rospy.Publisher('elfin_basic_api/cart_path_goal', PoseArray

    def pub_cart_path_goal(self):
        pa=PoseArray()
        pa.header.stamp=rospy.get_rostime()
        pa.header.frame_id='elfin_base_link'

        ps=Pose()
        ps.position.x=0.420
        ps.position.y=0.0
        ps.position.z=0.445
        ps.orientation.x=0
        ps.orientation.y=1
        ps.orientation.z=0
```

3.4.1.4. elfin_arm_controller/command

说明：向机械臂发送一条运动轨迹，消息类型为 trajectory_msgs/JointTrajectory，用户需要实现一个话题发布者向 elfin_arm_controller/command 发送一条运动轨迹，该轨迹可以是用户自行规划或通过 moveit 接口获取规划好的再向该接口发送轨迹。

C++示例：

elfin_path_command.cpp

```cpp
/*
Elfin-ROS控制接口示例
elfin_arm_controller/command
*/

#include <ros/ros.h>
#include <trajectory_msgs/JointTrajectory.h>
#include <trajectory_msgs/JointTrajectoryPoint.h>

int main(int argc, char *argv[])
{
    ros::init(argc, argv, "elfin_traj_command");
    ros::NodeHandle nh;
    ros::Publisher pub = nh.advertise<trajectory_msgs::JointTrajectory>("elfin_ar

    trajectory_msgs::JointTrajectory elfin_traj_path;
    trajectory_msgs::JointTrajectoryPoint elfin_traj_point;

    elfin_traj_path.joint_names = {"elfin_joint1", "elfin_joint2", "elfin_joint3"
                "elfin_joint4", "elfin_joint5", "elfin_joint6"};
    elfin_traj_path.header.stamp = ros::Time::now();

    elfin_traj_point.positions = {0.3,0.3,0.3,0.3,0.3,0.3};
    elfin_traj_point.velocities = {0.1,0.1,0.1,0.1,0.1,0.1};
    elfin_traj_point.accelerations = {0.2,0.2,0.2,0.2,0.2,0.2};
    elfin_traj_point.time_from_start = ros::Duration(0.5);
```

Python 示例：

elfin_path_command.py

```python
#!/usr/bin/env python2
# -*- coding: utf-8 -*-
"""
Elfin-ROS控制接口示例
elfin_arm_controller/command
"""

import rospy
from trajectory_msgs.msg import JointTrajectory, JointTrajectoryPoint

class Elfin_Traj_Command(object):
    def __init__(self):
        self.elfin_traj_pub = rospy.Publisher('elfin_arm_controller/command', JointTraj

    def pub_elfin_traj(self):
        elfin_joint_traj = JointTrajectory()
        elfin_joint_traj_point = JointTrajectoryPoint()

        elfin_joint_traj.joint_names = ['elfin_joint1', 'elfin_joint2', 'elfin_joint3',
                'elfin_joint4', 'elfin_joint5', 'elfin_joint6']
        elfin_joint_traj.header.stamp = rospy.get_rostime()

        elfin_joint_traj_point.positions = [0.3,0.3,0.3,0.3,0.3,0.3]
        elfin_joint_traj_point.velocities = [0.1,0.1,0.1,0.1,0.1,0.1]
        elfin_joint_traj_point.accelerations = [0.2,0.2,0.2,0.2,0.2,0.2]
        elfin_joint_traj_point.time_from_start = rospy.Duration.from_sec(0.5)
```

3.4.1.5. elfin_basic_api/stop_teleop

说明： 机械臂停止服务，呼叫本服务会令机械臂停止运动，用户需要向 elfin_basic_api/stop_teleop 服务端呼叫类型为 std_srvs/SetBool 的消息，本质上是向机械臂发送一条空轨迹，在成功呼叫本服务后会应答”stop moving ” 返回值。

C++示例：

stop_teleop_demo.cpp

```cpp
/*
Elfin-ROS控制接口示例
elfin_basic_api/stop_teleop
*/

#include <ros/ros.h>
#include <std_srvs/SetBool.h>

int main(int argc, char *argv[])
{
    ros::init(argc, argv, "stop_teleop_demo");
    ros::NodeHandle nh;
    ros::ServiceClient stop_teleop_demo = nh.serviceClient<std_srvs::SetBool>("elfin_basi
    std_srvs::SetBool::Request stop_teleop_requset;
    std_srvs::SetBool::Response stop_teleop_response;
    stop_teleop_requset.data = true;

    ros::Duration(1.0).sleep();
    stop_teleop_demo.call(stop_teleop_requset,stop_teleop_response);
    if(stop_teleop_response.success){
        std::cout<<stop_teleop_response.message<<std::endl;
    }
    ros::spin();
    return 0;
}
```

Python 示例：

stop_teleop_demo.py

```python
#!/usr/bin/env python2
# -*- coding: utf-8 -*-
"""
Elfin-ROS控制接口示例
elfin_basic_api/stop_teleop
"""

import rospy
from std_srvs.srv import SetBool, SetBoolRequest

class Stop_Teleop_Demo(object):
    def __init__(self):
        self.stop_teleop = rospy.ServiceProxy("elfin_basic_api/stop_teleop",SetBool)
        self.call_stop_teleop_req = SetBoolRequest()
        self.call_stop_teleop_req.data = True

    def call_elfin_stop_teleop(self):
        resp = self.stop_teleop.call(self.call_stop_teleop_req)
        if resp.success:
            print(resp.message)

if __name__=='__main__':
    rospy.init_node('server_call', anonymous=True)
    test=Stop_Teleop_Demo()
    rospy.sleep(1)
    test.call_elfin_stop_teleop()
    rospy.spin()
```

3.4.1.6. elfin_basic_api/enable_robot

说明： 机械臂使能服务，呼叫本服务会将机械臂从去使能状态切换到使能状态，用户需要向 elfin_basic_api/enable_robot 服务端呼叫类型为 std_srvs/SetBool 的消息。

C++示例：

elfin_enable_demo.cpp

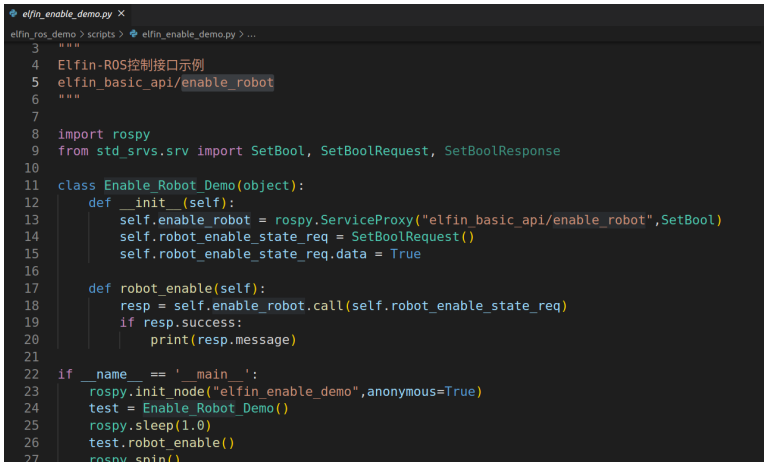
```cpp
/*
Elfin-ROS控制接口示例
elfin_basic_api/enable_robot
*/

#include <ros/ros.h>
#include <std_srvs/SetBool.h>
#include <std_srvs/SetBoolRequest.h>

int main(int argc, char *argv[])
{
    ros::init(argc, argv, "elfin_enable_demo_node");
    ros::NodeHandle nh;
    ros::ServiceClient enable_robot = nh.serviceClient<std_srvs::SetBool>("elfin_basic_a
    std_srvs::SetBool::Request enable_robot_requset;
    std_srvs::SetBool::Response enable_robot_response;
    enable_robot_requset.data = true;

    ros::Duration(1.0).sleep();
    enable_robot.call(enable_robot_requset, enable_robot_response);
    if(enable_robot_response.success)
    {
        std::cout<<enable_robot_response.message<<std::endl;
    }
    ros::spin();
    return 0;
}
```

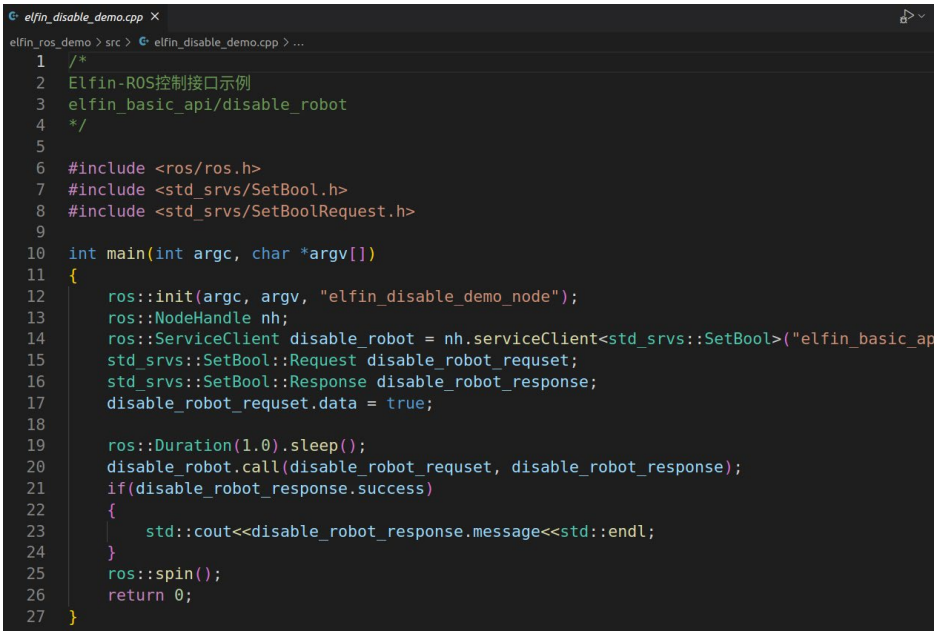
Python 示例：



3.4.1.7. elfin_basic_api/disable_robot

说明： 机械臂去使能服务，呼叫本服务会将机械臂从使能状态切换到去使能状态，用户需要向 elfin_basic_api/disable_robot 服务端呼叫类型为 std_srvs/SetBool 的消息。

C++示例：



Python 示例：

elfin_disable_demo.py

elfin_ros_demo > scripts > elfin_disable_demo.py > Disable_Robot_Demo > __init__

```python
"""
Elfin-ROS控制接口示例
elfin_basic_api/disable_robot
"""

import rospy
from std_srvs.srv import SetBool, SetBoolRequest, SetBoolResponse

class Disable_Robot_Demo(object):
    def __init__(self):
        self.disable_robot = rospy.ServiceProxy("elfin_basic_api/disable_robot",SetBool)
        self.robot_disable_state_req = SetBoolRequest()
        self.robot_disable_state_req.data = True

    def robot_disable(self):
        resp = self.disable_robot.call(self.robot_disable_state_req)
        if resp.success:
            print(resp.message)

if __name__ == '__main__':
    rospy.init_node("elfin_disable_demo",anonymous=True)
    test = Disable_Robot_Demo()
    rospy.sleep(1.0)
    test.robot_disable()
    rospy.spin()
```

3.4.2.参数/状态接口

3.4.2.1. elfin_arm_controller/state

说明： 订阅该话题可反映机械臂各个关节的状态，消息类型为 control_msgs/JointTrajectoryControllerState。

C++示例：

get_robot_state.cpp

elfin_ros_demo > src > get_robot_state.cpp > robot_state_cb(const control_msgs::JointTrajectoryControllerState::ConstPtr &)

```cpp
/*
Elfin-ROS控制接口示例
elfin_arm_controller/state
*/

#include <ros/ros.h>
#include <control_msgs/JointTrajectoryControllerState.h>

void robot_state_cb(const control_msgs::JointTrajectoryControllerState::ConstPtr& msg_p)
{
    if(msg_p != NULL){
        std::cout<<"joint_pos: "<<std::endl;
        for(int i =0;i<msg_p->actual.positions.size();i++)
        {
            std::cout<<"joint"<<i+1<<":"<<msg_p->actual.positions[i]<<",";
        }
        std::cout<<std::endl;
        std::cout<<"joint_vel: "<<std::endl;
        for(int i =0;i<msg_p->actual.velocities.size();i++)
        {
            std::cout<<"joint"<<i+1<<":"<<msg_p->actual.velocities[i]<<",";
        }
        std::cout<<std::endl;
        std::cout<<"----------------"<<std::endl;
    }
}
int main(int argc, char *argv[])
```

Python 示例：

```python
"""
Elfin-ROS控制接口示例
elfin_arm_controller/state
"""

import rospy
from control_msgs.msg import JointTrajectoryControllerState

class Get_Robot_State_Demo(object):
    def __init__(self):
        self.get_robot_state = rospy.Subscriber("elfin_arm_controller/state",JointTrajector

    def robot_state_cb(self,msg):
        joint_position = msg.actual.positions
        joint_vel = msg.actual.velocities

        print("joint position:\n joint1: %f,joint2: %f,joint3: %f,joint4: %f,joint5: %f,joi
            (joint_position[0],joint_position[1],joint_position[2],joint_position[3],joint_
        print("joint vel:\n joint1: %f,joint2: %f,joint3: %f,joint4: %f,joint5: %f,joint6:
            (joint_vel[0],joint_vel[1],joint_vel[2],joint_vel[3],joint_vel[4],joint_vel[5])
        print("------------------------------")

if __name__ == '__main__':
    rospy.init_node("get_robot_state", anonymous=True)
    test = Get_Robot_State_Demo()
    rospy.spin()
```

3.4.2.2. elfin_ros_control/elfin/enable_state

说明： 订阅该话题可反映机械臂当前使能状态，消息类型为 std_msgs/Bool，消息内容为 true 代表使能状态，为 false 代表去使能状态。

C++示例：

```cpp
/*
Elfin-ROS控制接口示例
elfin_ros_control/elfin/enable_state
*/

#include <ros/ros.h>
#include <std_msgs/Bool.h>

void enable_state_cb(const std_msgs::Bool::ConstPtr& msg_p){
    if(msg_p->data == false)
    {
        ROS_INFO("robot disable");
    }
    if(msg_p->data == true)
    {
        ROS_INFO("robot enable");
    }
}

int main(int argc, char *argv[])
{
    ros::init(argc, argv, "get_enable_state_node");
    ros::NodeHandle nh;
    ros::Subscriber sub = nh.subscribe<std_msgs::Bool>("/elfin_ros_control/elfin/enabl
    ros::spin();
    return 0;
}
```

Python 示例：

```python
#!/usr/bin/env python2
# -*- coding: utf-8 -*-
"""
Elfin-ROS控制接口示例
elfin_ros_control/elfin/enable_state
"""
import rospy
from std_msgs.msg import Bool

class Robot_Enable_State_Demo(object):
    def __init__(self):
        self.get_enable_state = rospy.Subscriber("elfin_ros_control/elfin/enable_state",B

    def enable_state_cb(self,msg):
        if msg.data == False:
            print("robot disable")
        if msg.data == True:
            print("robot enable")

if __name__ == '__main__':
    rospy.init_node('get_enable_state', anonymous=True)
    test=Robot_Enable_State_Demo()
    rospy.spin()
```

3.4.3.IO 接口

3.4.3.1. elfin_ros_control/elfin/io_port1/write_do

说明： 向 DO 写入内容服务，用户需要向 elfin_ros_control/elfin/io_port1/write_do 服务端呼叫类型为 elfin_robot_msgs/ElfinIODWrite 的消息，该消息为 Elfin-ROS 包中自定义的消息类型，具体内容可在 elfin_robot_msgs 功能包中查看。

C++示例：

write_do_demo.cpp

elfin_ros_demo > src > write_do_demo.cpp > ...

```cpp
/*
Elfin-ROS控制接口示例
elfin_ros_control/elfin/io_port1/write_do
*/

#include <ros/ros.h>
#include <elfin_robot_msgs/ElfinIODWrite.h>
#include <elfin_robot_msgs/ElfinIODWriteRequest.h>
#include <elfin_robot_msgs/ElfinIODWriteResponse.h>

int main(int argc, char* argv[])
{
    ros::init(argc, argv, "write_do_demo");
    ros::NodeHandle nh;
    ros::ServiceClient write_do_demo = nh.serviceClient<elfin_robot_msgs::Elfin

    elfin_robot_msgs::ElfinIODWrite::Request read_do_req;
    elfin_robot_msgs::ElfinIODWrite::Response read_do_resp;
    read_do_req.digital_output = 8192;

    ros::Duration(1.0).sleep();
    write_do_demo.call(read_do_req,read_do_resp);
    if(read_do_resp.success){
        std::cout<<"Write DO Success"<<std::endl;
    }

    ros::spin();
```

Python 示例：

write_do_demo.py

elfin_ros_demo > scripts > write_do_demo.py > ...

```python
#!/usr/bin/env python2
# -*- coding: utf-8 -*-
"""
Elfin-ROS控制接口示例
elfin_ros_control/elfin/io_port1/write_do
"""

import rospy
from elfin_robot_msgs.srv import ElfinIODWrite, ElfinIODWriteRequest
class Write_DO_Demo(object):
    def __init__(self):
        self.write_robot_do = rospy.ServiceProxy("elfin_ros_control/elfin/io_port1/wr
        self.write_robot_do_req = ElfinIODWriteRequest()
        self.write_robot_do_req.digital_output = 8192

    def call_elfin_write_do(self):
        resp = self.write_robot_do.call(self.write_robot_do_req)
        if resp.success:
            print("Write DO Success")

if __name__ == '__main__':
    rospy.init_node('write_do_demo', anonymous=True)
    test=Write_DO_Demo()
    rospy.sleep(1)
    test.call_elfin_write_do()
    rospy.spin()
```

3.4.3.2. elfin_ros_control/elfin/io_port1/read_di

说明： 读取 DI 内容服务，用户需要向 elfin_ros_control/elfin/io_port1/read_di 服务端呼叫类型为 elfin_robot_msgs/ElfinIODRead 的消息，该消息为 Elfin-ROS 包中自定义的消息类型，具体内容可在 elfin_robot_msgs 功能包中查看。

C++示例：

read_di_demo.cpp

elfin_ros_demo > src > read_di_demo.cpp > ...

```cpp
/*
Elfin-ROS控制接口示例
elfin_ros_control/elfin/io_port1/read_di
*/

#include <ros/ros.h>
#include "elfin_robot_msgs/ElfinIODRead.h"
#include "elfin_robot_msgs/ElfinIODReadRequest.h"
#include "elfin_robot_msgs/ElfinIODReadResponse.h"

int main(int argc, char* argv[])
{
    ros::init(argc, argv, "read_di_demo");
    ros::NodeHandle nh;
    ros::ServiceClient read_di_demo = nh.serviceClient<elfin_robot_msgs::ElfinIODRead>("e

    elfin_robot_msgs::ElfinIODRead::Request read_di_req;
    elfin_robot_msgs::ElfinIODRead::Response read_di_resp;
    read_di_req.data = true;

    ros::Duration(1.0).sleep();
    read_di_demo.call(read_di_req,read_di_resp);
    std::cout<<read_di_resp.digital_input<<std::endl;

    ros::spin();
    return 0;
}
```

Python 示例：

read_di_demo.py

elfin_ros_demo > scripts > read_di_demo.py > Read_DI_Demo > __init__

```python
#!/usr/bin/env python2
# -*- coding: utf-8 -*-
"""
Elfin-ROS控制接口示例
elfin_ros_control/elfin/io_port1/read_di
"""

import rospy
from elfin_robot_msgs.srv import ElfinIODRead,ElfinIODReadRequest
class Read_DI_Demo(object):
    def __init__(self):
        self.read_robot_di = rospy.ServiceProxy("elfin_ros_control/elfin/io_port1/read_
        self.read_robot_di_req = ElfinIODReadRequest()
        self.read_robot_di_req.data = True

    def call_elfin_read_DI(self):
        resp = self.read_robot_di.call(self.read_robot_di_req)
        print("Robot DI: ", resp.digital_input)

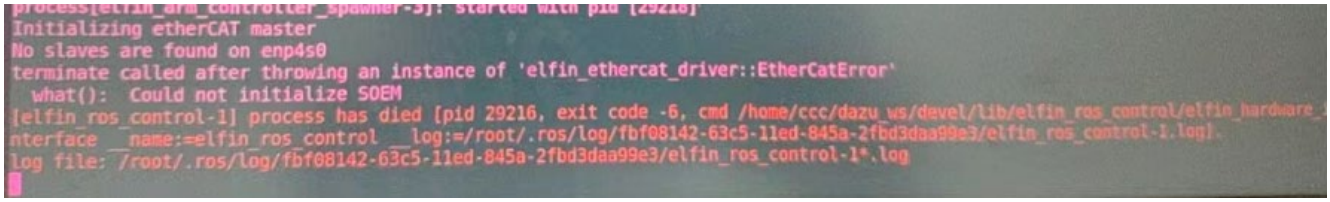
if __name__ == '__main__':
    rospy.init_node('read_di_demo', anonymous=True)
    test=Read_DI_Demo()
    rospy.sleep(1)
    test.call_elfin_read_DI()
    rospy.spin()
```

第四章 常见问题的排查与解决方法

4.1.连接问题

4.1.1.没有扫描到从站

表现：在运行 roslaunch elfin_robot_bringup elfin_ros_control.launch 的终端中出现如下图所示报错

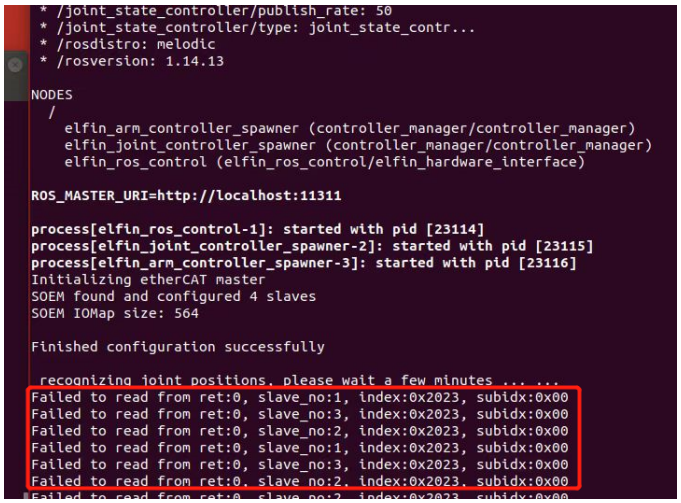


解决方案：

1. 请使用示教器查看机器人是否上电，如未上电请先将本体上电，可参考本文 3.2.3
2. 请查看当前网口名称是否设置正确，并且该网口不需要设置固定 IP
3. 请确保当前电脑与机器人处于直连状态，不可接入交换机

4.1.2.连接成功，但一直会有打印信息

表现：在运行 roslaunch elfin_robot_bringup elfin_ros_control.launch 的终端中出现如下图所示的打印信息



解决方案：

1. 请检查当前使用的分支与机器人末端类型是否匹配
2. 请使用与末端类型相匹配的分支重启启动